

Clarus Analysis Report

Document: rfc8446.txt

Analyzed: 2026-05-08T23:34:20.731158

1. Document Processing

Metric	Value
Total Segments	3714
Normative Segments	537
Informative Segments	2070
Unknown Segments	1107
High Confidence (≥ 0.7)	725
Low Confidence (< 0.4)	0
Text Length (chars)	346699

2. Semantic Extraction

Metric	Value
Total Profiles	49
Average Confidence	70.4%

Profiles (49)

#	Sentence	Subject	Object	Modality	Condition
1	If the server selects a PSK, then it MUST also select a key	it	also select a key	MUST	the server selects a PSK
2	When a client first connects to a server, it is REQUIRED to send the	it	the	is REQUIRED to	a client first connects to a server
3	In the event that a client requests additional functionality using extensions and this functionality is not supplied by the server, the client MAY abort the handshake.	client	abort the handshake	MAY	a client requests additional functionality using extensions and this functionality is not supplied by the server
4	below. If a match is found, the client MUST abort the handshake with	match	abort the handshake with	MUST	a match is found
5	message if it is able to find an acceptable set of parameters but the	it	of parameters but the	is able to	it is able to find an acceptable set of parameters but the

6	HRR (HelloRetryRequest). If an implementation receives an extension which it recognizes and which is not specified for the message in which it appears, it MUST abort the handshake with an	it	abort the handshake with an	MUST	an implementation receives an extension which it recognizes and which is not specified for the message in which it appears
7	When multiple extensions of different types are present, the extensions MAY appear in any order, with the exception of "pre_shared_key" (Section 4.2.11) which MUST be the last extension in the ClientHello (but can appear anywhere in the ServerHello	extensions	appear in any order	MAY	multiple extensions of different types are present
8	sufficient, but extreme care is needed when the extension changes the meaning of messages sent in the handshake phase. Designers and implementors should be aware of the fact that until the handshake has been authenticated, active attackers can modify messages and insert, remove, or replace extensions.	handshake	modify messages and insert	should	the extension changes the meaning of messages sent in the handshake phase
9	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present	any unknown versions that	use the ClientHello	MUST NOT	this extension is present in the ClientHello
10	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the	If this extension	ignore the	MUST	this extension is present
11	ClientHello). When sending the new ClientHello, the client MUST copy	client	copy	MUST	sending the new ClientHello
12	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.	it	ignore these records	MUST	it were the first message to be received
13	it MUST use the RSASSA-PSS OID [RFC5756]. When used in certificate signatures, the algorithm parameters MUST be DER	it	DER	MUST	used in certificate signatures
14	the same order. However, the values MAY be a non-contiguous subset of the "supported_groups" extension and MAY omit the most preferred groups. Such a situation could arise if the most preferred groups	values	omit the most preferred groups	MAY	the most preferred groups
15	If the server rejects the "early_data" extension, the client application MAY opt to retransmit the Application Data previously sent in early data once the handshake has been completed. Note that automatic retransmission of early data could result in incorrect assumptions regarding the status of the connection. For instance, when the negotiated connection selects a different ALPN protocol from what was used for the early data, an application might need to	client application	need to	MAY	the server rejects the "early_data" extension
16	A TLS implementation SHOULD NOT automatically resend early data; applications are in a better position to decide when retransmission	TLS implementation	automatically resend early data	SHOULD	retransmission
17	not in the PSK database or encrypted with an unknown key) SHOULD simply be ignored. If no acceptable PSKs are found, the server	no acceptable PSKs	simply be ignored	SHOULD	no acceptable PSKs are found
18	If the server supplies an "early_data" extension, the client MUST verify that the server's selected_identity is 0. If any other value is returned, the client MUST abort the handshake with an	any other value	abort the handshake with an	MUST	the server supplies an "early_data" extension
19	certificates. The client MUST check EncryptedExtensions for the presence of any forbidden extensions and if any are found MUST abort	any are found	abort	MUST	any are found MUST abort

20	if defined for this message. Clients MUST ignore unrecognized	Clients	ignore unrecognized	MUST	defined for this message
21	extensions in the CertificateRequest message from the server. If an extension applies to the entire chain, it SHOULD be included in	it	included in	SHOULD	an extension applies to the entire chain
22	independently, a certificate that specifies a trust anchor MAY be omitted from the chain, provided that supported peers are known to	trust anchor	omitted from the chain	MAY	supported peers are known to
23	- The certificate type MUST be X.509v3 [RFC5280], unless explicitly	certificate type	X	MUST	explicitly
24	algorithm advertised by the client if it is able to provide such a chain (see Section 4.2.3). Certificates that are self-signed or certificates that are expected to be trust anchors are not validated as part of the chain and therefore MAY be signed with any algorithm.	it	signed with any algorithm	is able to	it is able to provide such a chain (see Section 4
25	If the server cannot produce a certificate chain that is signed only via the indicated supported algorithms, then it SHOULD continue the	it	continue the	SHOULD	the server cannot produce a certificate chain that is signed only via the indicated supported algorithms
26	- The certificate type MUST be X.509v3 [RFC5280], unless explicitly	certificate type	X	MUST	explicitly
27	up to this point. Servers MUST send this message when authenticating via a certificate. Clients MUST send this message whenever	Servers	send this message whenever	MUST	authenticating via a certificate
28	is non-empty). When sent, this message MUST appear immediately after	this message	appear immediately after	MUST	sent
29	If the verification fails, the receiver MUST terminate the handshake	receiver	terminate the handshake	MUST	the verification fails
30	Recipients of Finished messages MUST verify that the contents are correct and if incorrect MUST terminate the connection with a	contents	terminate the connection with a	MUST	incorrect MUST terminate the connection with a
31	in that case would waste a single-use ticket. If such an indication is provided (externally or by any other means), clients MAY resume with a different SNI value.	indication	ticket	MAY	such an indication is provided (externally or by any other means)
32	max_early_data_size: The maximum amount of 0-RTT data that the client is allowed to send when using this ticket, in bytes. Only	client	when using this ticket	is allowed to	using this ticket
33	and Finished. If it declines, it MUST send a Certificate message	it	send a Certificate message	MUST	it declines
34	request_update: Indicates whether the recipient of the KeyUpdate should respond with its own KeyUpdate. If an implementation	KeyUpdate	respond with its own KeyUpdate	should	an implementation
35	MUST verify that all messages immediately preceding a key change align with a record boundary; if not, then they MUST terminate the	they	terminate the	MUST	not
36	If the decryption fails, the receiver MUST terminate the connection	receiver	terminate the connection	MUST	the decryption fails
37	When generating a TLSCiphertext record, implementations MAY choose to pad. An unpadded record is just a record with a padding length of zero. Padding is a string of zero-valued bytes appended to the	unpadded record	choose to pad	MAY	generating a TLSCiphertext record
38	TLSCiphertext.content if the sender desires. This permits generation of plausibly sized cover traffic in contexts where the presence or absence of activity may be sensitive. Implementations	absence of activity	sensitive	may	the sender desires
39	connections, with the exception of the PSKs associated with session tickets, which SHOULD be discarded if possible.	which	discarded if possible	SHOULD	possible

40	user_canceled: This alert notifies the recipient that the sender is canceling the handshake for some reason unrelated to a protocol failure. If a user cancels an operation after the handshake is complete, just closing the connection by sending a "close_notify" is more appropriate. This alert SHOULD be followed by a	sender	followed by a	SHOULD	a user cancels an operation after the handshake is complete
41	alert has been received MUST be ignored. If a transport-level close	has been received	ignored	MUST	a transport-level close
42	deployments will maintain state at that level. Therefore, in normal operation, clients will not know which, if any, of these mechanisms servers actually implement and hence MUST only send early data which	deployments	maintain state at that level	MUST	any
43	In addition to the direct effects of replays, there is a class of attacks where even operations normally considered idempotent could be exploited by a large number of replays (timing attacks, resource limit exhaustion and others, as described in Appendix E.5). Those can be mitigated by ensuring that every 0-RTT payload can be replayed	there	replayed	can	even operations normally considered idempotent could be exploited by a large number of replays (timing attacks)
44	All implementations MUST send and use these extensions when offering	All implementations	se extensions when offering	MUST	offering
45	- If not containing a "pre_shared_key" extension, it MUST contain	it	contain	MUST	not containing a "pre_shared_key" extension
46	- If containing a "supported_groups" extension, it MUST also contain	it	also contain	MUST	containing a "supported_groups" extension
47	Note that with the transitions as shown above, clients may send alerts that derive from post-ServerHello messages in the clear or with the early data keys. If clients need to send such alerts, they	they	such alerts	may	clients need to send such alerts
48	The exporter_master_secret and early_exporter_master_secret are derived to be independent of the traffic keys and therefore do not represent a threat to the security of traffic encrypted with those keys. However, because these secrets can be used to compute any exporter value, they SHOULD be erased as soon as possible. If the total set of exporter labels is known, then implementations SHOULD	they	erased as soon as possible	can	the total set of exporter labels is known
49	Replayable 0-RTT data presents a number of security threats to TLS- using applications, unless those applications are specifically engineered to be safe under replay (minimally, this means idempotent, but in many cases may also require other stronger conditions, such as constant-time response). Potential attacks include:	unless those applications	also require other stronger conditions	may	those applications are specifically engineered to be safe under replay (minimally)

3. Terminology Analysis

Metric	Value
Defined Terms	2
Undefined Terms	48
Definition Coverage	4.0%

Defined Terms (2)

Term	Definition / Context
RSASSA-PSS	used with RSA cipher suites
Derive-Secret	called with four distinct transcripts; in a

Undefined Terms — FAIL-001 (48)

Code	Term	Context Excerpt
FAIL-001	PSK-FINISHED	... The TLS 1.3 record layer without elision", 2018, < https://eprint.iacr.org/2018/634 >. [PSK-FINISHED] Scott, S., Cremers, C., Horvat, M., and T. van der Merwe, "Revision 10: possible attack if client authentication is allowed during PSK",...
FAIL-001	PSK 0-RTT	This improves security for all 0-RTT data and PSK usage when PSK is used without (EC)DHE. Because this mechanism requires sharing the session database between server nodes in environments with multiple distributed servers, it may be hard to achieve high rates of successful PSK 0-RTT conn...
FAIL-001	TLS 1.3	This document defines TLS version 1.3. While TLS 1.3 is not directly compatible with previous versions, all versions of TLS incorporate a versioning mechanism which allows clients and servers to interoperably negotiate a common version if one is supported by both peers. This document...
FAIL-001	TLS 1.0	This document describes TLS 1.3, which uses the version 0x0304. This version value is historical, deriving from the use of 0x0301 for TLS 1.0 and 0x0300 for SSL 3.0. In order to maximize backward compatibility, a record containing an initial ClientHello SHOULD h...
FAIL-001	Royal Holloway	... Consultant david.hopwood@blueyonder.co.uk Rescorla Standards Track [Page 155] RFC 8446 TLS August 2018 Marko Horvat MPI-SWS mhorvat@mpi-sws.org Jonathan Hoyland Royal Holloway, Universi...
FAIL-001	RFC 8439	... DOI 10.17487/RFC8174, May 2017, < https://www.rfc-editor.org/info/rfc8174 >. [RFC8439] Nir, Y. and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols", RFC 8439, DOI 10.17487/RFC8439, June 2018, < https://www.rfc-editor.org/info/rfc8439 >. [...
FAIL-001	ISO/IEC 8825-1:2015	... of Standards and Technology report, DOI 10.6028/NIST.FIPS.180-4, August 2015. [X690] ITU-T, "Information technology -- ASN.1 encoding rules: Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) and Distinguished Encod...
FAIL-001	ANSI ANS X9.62-2005	...al Standards Institute, "Public Key Cryptography for the Financial Services Industry: The Elliptic Curve Digital Signature Algorithm (ECDSA)", ANSI ANS X9.62-2005, November 2005. [GCM] Dworkin, M., "Recommendation for Block Cipher Modes of ...
FAIL-001	TLS 1.1 and 1.2	...Canetti IBM canetti@watson.ibm.com Rescorla Standards Track [Page 153] RFC 8446 TLS August 2018 Matt Caswell OpenSSL matt@openssl.org Stephen Checkoway University of Illinois at ...
FAIL-001	TLS ContentType	... an unexpected record type, it MUST terminate the connection with an "unexpected_message" alert. New record content type values are assigned by IANA in the TLS ContentType registry as described in Section 11. 5.1. Record Layer The record layer fragments information blocks int...
FAIL-001	Firefox TLS 1.3	... Techniques", ASIACRYPT 2000, DOI 10.1007/3-540-44448-3_42, October 2000. [Res17a] Rescorla, E., "Preliminary data on Firefox TLS 1.3 Middlebox experiment", message to the TLS mailing list, 5 December 2017, < https://www.ietf.org/ ...
FAIL-001	Derive-Secret's	This produces a full key derivation schedule shown in the diagram below. In this diagram, the following formatting conventions apply: - HKDF-Extract is drawn as taking the Salt argument from the top and the IKM argument from the left, with its output to the bottom and the na...
FAIL-001	ContentType +	—
FAIL-001	NCC Group	Victor Vasiliev Google vasilvv@google.com Hoeteck Wee Ecole Normale Superieure, Paris hoeteck@alum.mit.edu Tom Weinstein David Wong NCC Group david.wong@nccgroup.trust Christopher A. Wood Apple Inc. cawood@apple.com Tim Wright Vodafone timoth...
FAIL-001	DoS	...to TLS 1.3, the client MUST abort the handshake with an "illegal_parameter" alert. 4.2.2. Cookie struct { opaque cookie<1..2^16-1>; } Cookie; Cookies serve two primary purposes: - Allowing the server to force the client to demonstrate reac...
FAIL-001	Peter Wu	...CC Group david.wong@nccgroup.trust Christopher A. Wood Apple Inc. cawood@apple.com Tim Wright Vodafone timothy.wright@vodafone.com Peter Wu Independent peter@lekensteyn.nl Kazu Yamamoto Internet Initiative Japan Inc. kazu@ij.ad.jp Author's...
FAIL-001	Shared Secret Calculation	Updating Traffic Secrets94 7.3. Traffic Key Calculation95 7.4. (EC)DHE Shared Secret Calculation95 7.4.1. Finite Field Diffie-Hellman95 7.4....

FAIL-001	Record Payload Protection	Record Layer78 5.2. Record Payload Protection80 5.3. Per-Record Nonce82 5.4.
FAIL-001	Updating Traffic Secrets	Key Schedule91 7.2. Updating Traffic Secrets94 7.3. Traffic Key Calculation95 7.4. (EC)DHE Shared Secret Calc...
FAIL-001	Johns Hopkins University	...et Microsoft fournet@microsoft.com Rescorla Standards Track [Page 154] RFC 8446 TLS August 2018 Anil Gangolli anil@busybuddha.org David M. Garrett dave@nullreference.com Illy...
FAIL-001	Post-Compromise Security	146 E.1.3. 0-RTT146 E.1.4. Exporter Independence146 E.1.5. Post-Compromise Security146 E.1.6. External References
FAIL-001	Traffic Key Calculation	Updating Traffic Secrets94 7.3. Traffic Key Calculation95 7.4. (EC)DHE Shared Secret Calculation95 7.4.1. Finite Field Diffie-Hellman95 7.4....
FAIL-001	Implementation Pitfalls	Certificates and Authentication135 C.3. Implementation Pitfalls135 C.4. Client Tracking Prevention137 C.5.
FAIL-001	Specification Required	IANA has updated these to reference this document. The registries and their allocation policies are below: - TLS Cipher Suites registry: values with the first byte in the range 0-254 (decimal) are assigned via Specification Required [RFC8126].
FAIL-001	Server Name Indication	Mandatory-to-Implement Extensions In the absence of an application profile standard specifying otherwise, a TLS-compliant application MUST implement the following TLS extensions: - Supported Versions ("supported_versions"; Section 4.2.1) - Cookie ("cookie"; Section 4.2.2) ...
FAIL-001	Offensive Technologies	...ion Security, Vol. 2016, DOI 10.1186/s13635-016-0030-7, February 2016. [HGFS15] Hlauschek, C., Gruber, M., Fankhauser, F., and C. Schanes, "Prying Open Pandora's Box: KCI Attacks against TLS", Proceedings of USENIX Workshop on Offensiv...
FAIL-001	Early Data Indication	Pre-Shared Key Exchange Modes51 4.2.10. Early Data Indication52 4.2.11. Pre-Shared Key Extension55 4.3.
FAIL-001	Client Authentication	OID Filters45 4.2.6. Post-Handshake Client Authentication47 4.2.7. Supported Groups47 4.2.8.
FAIL-001	Initiative Japan Inc.	—
FAIL-001	Elliptic Curve Groups	...roups which the client supports for key exchange, ordered from most preferred to least preferred. Note: In versions of TLS prior to TLS 1.3, this extension was named "elliptic_curves" and only contained elliptic curve groups. See [RFC8422] and [RFC7919].
FAIL-001	Derive-Secret(Secret	...well as the functions defined below: HKDF-Expand-Label(Secret, Label, Context, Length) = HKDF-Expand(Secret, HkdfLabel, Length) Where HkdfLabel is specified as: struct { uint16 length = Length; opaque label<7..255> = "tls13 " ...
FAIL-001	Ticket Establishment	Authentication Messages132 B.3.4. Ticket Establishment132 B.3.5. Updating Keys133 B.4.
FAIL-001	Normative References	References109 12.1. Normative References109 12.2. Informative References112 Appendix A. State Machine
FAIL-001	External References	146 E.1.5. Post-Compromise Security146 E.1.6. External References147 E.2. Record Layer147 E.2.1.
FAIL-001	Alacrity Ventures	Rescorla Standards Track [Page 152] RFC 8446 TLS August 2018 Contributors Martin Abadi University of California, Santa Cruz abadi@cs.ucsc.edu Christopher Allen (co-editor of TLS 1.0) Alacrity Ven...
FAIL-001	Major Differences	Conventions and Terminology7 1.2. Major Differences from TLS 1.28 1.3. Updates Affecting TLS 1.29 2.
FAIL-001	Traffic Analysis	External References148 E.3. Traffic Analysis148 E.4. Side-Channel Attacks149 E.5.
FAIL-001	Badertscher, C.,	—
FAIL-001	Application Data	...e that remain are all Authenticated Encryption with Associated Data (AEAD) algorithms. The cipher suite concept has been changed to separate the authentication and key exchange mechanisms from the record protection algorithm (including secret key length) and a h...
FAIL-001	Cedric Fournet	...Canetti IBM canetti@watson.ibm.com Rescorla Standards Track [Page 153] RFC 8446 TLS August 2018 Matt Caswell OpenSSL matt@openssl.org Stephen Checkoway University of Illinois at ...
FAIL-001	November 2007	...iptic Curve Digital Signature Algorithm (ECDSA)", ANSI ANS X9.62-2005, November 2005. [GCM] Dworkin, M., "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC", NIST Special Publication 800-38D, DOI...

FAIL-001	December 2017	... < https://datatracker.ietf.org/meeting/100/materials/slides-100-tls-sessa-tls13/ >. [Ben17b] Benjamin, D., "Additional TLS 1.3 results from Chrome", message to the TLS mailing list, 18 December 2017, < https://www.ietf.org/mail-archive/web/tls/curre...
FAIL-001	November 2015	... of Standards and Technology report, DOI 10.6028/NIST.FIPS.180-4, August 2015. [X690] ITU-T, "Information technology -- ASN.1 encoding rules: Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) and Distinguished Encod...
FAIL-001	November 1976	References 12.1. Normative References [DH76] Diffie, W. and M. Hellman, "New directions in cryptography", IEEE Transactions on Information Theory, Vol. 22 No. 6, pp. 644-654, DOI 10.1109/TIT.1976.1055638, November 1976. [ECDSA] American Na...
FAIL-001	Eric Rescorla	...CC Group david.wong@nccgroup.trust Christopher A. Wood Apple Inc. cawood@apple.com Tim Wright Vodafone timothy.wright@vodafone.com Peter Wu Independent peter@lekensteyn.nl Kazu Yamamoto Internet Initiative Japan Inc. kazu@ij.ad.jp Author's...
FAIL-001	Taher Elgamal	...Canetti IBM canetti@watson.ibm.com Rescorla Standards Track [Page 153] RFC 8446 TLS August 2018 Matt Caswell OpenSSL matt@openssl.org Stephen Checkoway University of Illinois at ...
FAIL-001	November 2016	...C 7919, DOI 10.17487/RFC7919, August 2016, < https://www.rfc-editor.org/info/rfc7919 >. [RFC8017] Moriarty, K., Ed., Kaliski, B., Jonsson, J., and A. Rusch, "PKCS #1: RSA Cryptography Specifications Version 2.2", RFC 8017, DOI 10.17487/RFC8017, November...
FAIL-001	November 2005	...al Standards Institute, "Public Key Cryptography for the Financial Services Industry: The Elliptic Curve Digital Signature Algorithm (ECDSA)", ANSI ANS X9.62-2005, November 2005. [GCM] Dworkin, M., "Recommendation for Block Cipher Modes of ...

4. Contradiction Analysis

Total contradictions detected: **899**

#	Code	Type	Conf.	Statement A	Statement B
1	FAIL-003	semantic	100%	MUST send these messages in alignment with a record boundary.	field is deprecated and MUST be ignored for all purposes.
2	FAIL-003	semantic	100%	0x0303 (TLS 1.2). Clients MUST check for this extension prior to processing the rest of the ServerHello (although they will have to	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.
3	FAIL-003	semantic	100%	to determine the CSPRNG internal state and thereby predict the keying	support a compatible (EC)DHE group but do not predict it in the
4	FAIL-003	semantic	100%	this field MUST be non-empty, so a client not offering a pre-TLS 1.3 session MUST generate a new 32-byte value. This value	does not recognize, support, or wish to use, the server MUST ignore those cipher suites and process the remaining ones as
5	FAIL-003	semantic	100%	messages for a given response MUST appear consecutively on the wire	receives any other value, it MUST terminate the connection with an
6	FAIL-003	semantic	100%	A server which negotiates a version of TLS prior to TLS 1.3 MUST set	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.
7	FAIL-003	semantic	100%	this field MUST be non-empty, so a client not offering a pre-TLS 1.3 session MUST generate a new 32-byte value. This value	MUST contain exactly one byte, set to zero, which corresponds to
8	FAIL-003	semantic	100%	The receiver of a CertificateVerify message MUST verify the signature	the client MUST send an EndOfEarlyData message after receiving the
9	FAIL-003	semantic	100%	response to a ClientHello MUST set the last 8 bytes of their Random	process the extensions, starting with determining the version using "supported_versions". Clients MUST abort the handshake with an
10	FAIL-003	semantic	100%	The client MUST send a Certificate message if and only if the server	field having length 0). A Finished message MUST be sent regardless
11	FAIL-003	semantic	100%	is expected that TLS implementations will rely on their PKI libraries to perform certificate selection using certificate extension OIDs.	extension to the client. Clients MUST NOT act upon any information
12	FAIL-003	semantic	100%	support TLS 1.3 will send a normal TLS 1.3 ClientHello containing	not support TLS 1.3, it will respond with a ServerHello containing an

13	FAIL-003	semantic	100%	MUST be specified, and other extensions may optionally be included	field having length 0). A Finished message MUST be sent regardless
14	FAIL-003	semantic	100%	alert has been received MUST be ignored. If a transport-level close	receipt of a fatal alert message, both parties MUST immediately close
15	FAIL-003	semantic	100%	of [RFC6066]), with the server being required to enforce that the SNI value associated with the session matches the one specified in the	value is not present or does not validate, the server MUST abort the
16	FAIL-003	semantic	100%	Upon receipt of a HelloRetryRequest, the client MUST check the	process the extensions, starting with determining the version using "supported_versions". Clients MUST abort the handshake with an
17	FAIL-003	semantic	100%	guarantee that the 0-RTT data is accepted at most once. Servers	assumption that the server will not verify it (as recommended by
18	FAIL-003	semantic	100%	0x0303 (TLS 1.2). Clients MUST check for this extension prior to processing the rest of the ServerHello (although they will have to	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the
19	FAIL-003	semantic	100%	- The certificate MUST allow the key to be used for signing (i.e., the digitalSignature bit MUST be set if the Key Usage extension is	validate using any signature algorithm using an MD5 hash MUST abort
20	FAIL-003	semantic	100%	- The certificates MUST be signed using an acceptable signature	validate using any signature algorithm using an MD5 hash MUST abort
21	FAIL-003	semantic	100%	MUST verify that all messages immediately preceding a key change align with a record boundary; if not, then they MUST terminate the	field is deprecated and MUST be ignored for all purposes.
22	FAIL-003	semantic	100%	The server MUST send a Certificate message whenever the agreed-upon	The client MUST send a Certificate message if and only if the server
23	FAIL-003	semantic	100%	versions of TLS SHOULD support TLS 1.2. Servers MUST be prepared to receive ClientHellos that include this extension but do not include	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.
24	FAIL-003	semantic	100%	ClientHello). When sending the new ClientHello, the client MUST copy	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.
25	FAIL-003	semantic	100%	Any ticket MUST only be resumed with a cipher suite that has the same	transmitted by the client. The server MUST generate a fresh value
26	FAIL-003	semantic	100%	The receiver of a CertificateVerify message MUST verify the signature	Any ticket MUST only be resumed with a cipher suite that has the same
27	FAIL-003	semantic	100%	encoded vector must be an exact multiple of the length of a single	handshake message in an unexpected order MUST abort the handshake
28	FAIL-003	semantic	100%	TLS 1.3 and receives a ClientHello at any other time, it MUST terminate the connection with an "unexpected_message" alert.	does not recognize, support, or wish to use, the server MUST ignore those cipher suites and process the remaining ones as
29	FAIL-003	semantic	100%	If the server selects a PSK, then it MUST also select a key	If the server successfully selects parameters and does not require a
30	FAIL-003	semantic	100%	suitable certificate is available, the client MUST send a Certificate	field having length 0). A Finished message MUST be sent regardless
31	FAIL-003	semantic	100%	All certificates provided by the server MUST be signed by a signature	validate using any signature algorithm using an MD5 hash MUST abort
32	FAIL-003	semantic	100%	available within the environment. The selector must be an enumerated	handshake message in an unexpected order MUST abort the handshake
33	FAIL-003	semantic	100%	implementations MUST use the value sent in the resumption ClientHello	transmitted by the client. The server MUST generate a fresh value
34	FAIL-003	semantic	100%	- The certificate type MUST be X.509v3 [RFC5280], unless explicitly	validate using any signature algorithm using an MD5 hash MUST abort
35	FAIL-003	semantic	100%	- The certificate type MUST be X.509v3 [RFC5280], unless explicitly	validate using any signature algorithm using an MD5 hash MUST abort

36	FAIL-003	semantic	100%	versions of TLS SHOULD support TLS 1.2. Servers MUST be prepared to receive ClientHellos that include this extension but do not include	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the
37	FAIL-003	semantic	100%	this specification and which also support TLS 1.2 MUST negotiate	TLS 1.3, the client MUST abort the handshake with an
38	FAIL-003	semantic	100%	MUST NOT send Handshake and Alert records that have a zero-length	enforces all-zero padding octets, which allows for quick detection of
39	FAIL-003	semantic	100%	ClientHello as its first TLS message. The client will also send a	TLS and receives a TLS 1.3 ClientHello in a renegotiation, it MUST retain the previous protocol version. In particular, it MUST NOT negotiate TLS 1.3.
40	FAIL-003	semantic	100%	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.	extension, then the server MUST abort the handshake with a
41	FAIL-003	semantic	100%	length of the AEAD output will generally be larger than the	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion
42	FAIL-003	semantic	100%	Any ticket MUST only be resumed with a cipher suite that has the same	message, a different PSK will be derived for each ticket.
43	FAIL-003	semantic	100%	The server MUST send a Certificate message whenever the agreed-upon	suitable certificate is available, the client MUST send a Certificate
44	FAIL-003	semantic	100%	- A client sending a ClientHello MUST support all parameters	- A server receiving a ClientHello MUST correctly ignore all
45	FAIL-003	semantic	100%	In order to accept early data, the server MUST have accepted a PSK	rejects it, the server will generally not have the 0-RTT record protection keys and must instead use trial decryption (either with
46	FAIL-003	semantic	100%	this specification and which also support TLS 1.2 MUST negotiate	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the
47	FAIL-003	semantic	100%	handshake message in an unexpected order MUST abort the handshake	HelloRetryRequest. In that case, the client MUST send the same
48	FAIL-003	semantic	100%	transmitted by the client. The server MUST generate a fresh value	receives any other value, it MUST terminate the connection with an
49	FAIL-003	semantic	100%	alert has been received MUST be ignored. If a transport-level close	SHOULD send an appropriate fatal alert and MUST close the connection
50	FAIL-003	semantic	100%	Prior to accepting PSK key establishment, the server MUST validate	value is not present or does not validate, the server MUST abort the
51	FAIL-003	semantic	100%	If the server successfully selects parameters and does not require a	When a client first connects to a server, it is REQUIRED to send the
52	FAIL-003	semantic	100%	All certificates provided by the server MUST be signed by a signature	up to this point. Servers MUST send this message when authenticating via a certificate. Clients MUST send this message whenever
53	FAIL-003	semantic	100%	MUST contain exactly one byte, set to zero, which corresponds to	Appendix C for additional information. The last 8 bytes MUST be
54	FAIL-003	semantic	100%	certificate request and which will be echoed in the client's	Servers which are authenticating with a PSK MUST NOT send the
55	FAIL-003	semantic	100%	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.	present, then the parameters in the signature MUST be identical to
56	FAIL-003	semantic	100%	close_notify: This alert notifies the recipient that the sender will not send any more messages on this connection. Any data received after a closure alert has been received MUST be ignored.	receipt of a fatal alert message, both parties MUST immediately close
57	FAIL-003	semantic	100%	- The certificates MUST be signed using an acceptable signature	up to this point. Servers MUST send this message when authenticating via a certificate. Clients MUST send this message whenever
58	FAIL-003	semantic	100%	This document specifies version 1.3 of the Transport Layer Security	not specify how protocols add security with TLS; how to initiate TLS

59	FAIL-003	semantic	100%	HelloRetryRequest. In that case, the client MUST send the same	does not recognize, support, or wish to use, the server MUST ignore those cipher suites and process the remaining ones as
60	FAIL-003	semantic	100%	Implementations of this specification MUST send this extension in the	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the
61	FAIL-003	semantic	100%	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the	ClientHello). When sending the new ClientHello, the client MUST copy
62	FAIL-003	semantic	100%	alert has been received MUST be ignored. If a transport-level close	Each party MUST send a "close_notify" alert before closing its write
63	FAIL-003	semantic	100%	support a compatible (EC)DHE group but do not predict it in the	not support TLS 1.3, it will respond with a ServerHello containing an
64	FAIL-003	semantic	100%	- The certificate MUST allow the key to be used for signing (i.e., the digitalSignature bit MUST be set if the Key Usage extension is	up to this point. Servers MUST send this message when authenticating via a certificate. Clients MUST send this message whenever
65	FAIL-003	semantic	100%	not be able to distinguish these cases from allowed cases).	MUST send these messages in alignment with a record boundary.
66	FAIL-003	semantic	100%	Appendix C for additional information. The last 8 bytes MUST be	it sent in the ClientHello MUST abort the handshake with an
67	FAIL-003	semantic	100%	if previous versions of TLS are allowed to be negotiated, they MUST be present as well).	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the
68	FAIL-003	semantic	100%	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the	The length of the Salt MUST be equal to the length of the output
69	FAIL-003	semantic	100%	and receive Application Data over the connection. There are two	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This
70	FAIL-003	semantic	100%	- When (EC)DHE is in use, the server will also provide a "key_share"	Because TLS 1.3 forbids renegotiation, if a server has negotiated
71	FAIL-003	semantic	100%	extension, then the server MUST abort the handshake with a	it MUST use the RSASSA-PSS OID [RFC5756]. When used in certificate signatures, the algorithm parameters MUST be DER
72	FAIL-003	semantic	100%	- The certificate type MUST be X.509v3 [RFC5280], unless explicitly	up to this point. Servers MUST send this message when authenticating via a certificate. Clients MUST send this message whenever
73	FAIL-003	semantic	100%	certificates and decides to abort the handshake, then it MUST abort	up to this point. Servers MUST send this message when authenticating via a certificate. Clients MUST send this message whenever
74	FAIL-003	semantic	100%	up to this point. Servers MUST send this message when authenticating via a certificate. Clients MUST send this message whenever	Recipients of Finished messages MUST verify that the contents are correct and if incorrect MUST terminate the connection with a
75	FAIL-003	semantic	100%	server to authenticate itself via a certificate MUST send the	it MUST use the RSASSA-PSS OID [RFC5756]. When used in certificate signatures, the algorithm parameters MUST be DER
76	FAIL-003	semantic	100%	servers MUST be prepared for some delay, including receiving an	receives any other value, it MUST terminate the connection with an
77	FAIL-003	semantic	100%	Appendix C for additional information. The last 8 bytes MUST be	cipher suite that was not offered MUST abort the handshake with an
78	FAIL-003	semantic	100%	MUST NOT be used in any signatures of CertificateVerify messages.	handshake messages and are not included in the hash computations.
79	FAIL-003	semantic	100%	The client and the server must share knowledge that the connection is	alert has been received MUST be ignored. If a transport-level close
80	FAIL-003	semantic	100%	The server's certificate_list MUST always be non-empty. A client will send an empty certificate_list if it does not have an	response, the body of its "status_request" extension MUST be a

81	FAIL-003	semantic	100%	support TLS 1.3 will send a normal TLS 1.3 ClientHello containing	will respond to the ClientHello with an older ServerHello, but it will not correctly skip the 0-RTT data and will fail to complete the
82	FAIL-003	semantic	100%	When a client first connects to a server, it is REQUIRED to send the	does not recognize, support, or wish to use, the server MUST ignore those cipher suites and process the remaining ones as
83	FAIL-003	semantic	100%	T1 f1 = 8; /* T.f1 must always be 8 */	handshake message in an unexpected order MUST abort the handshake
84	FAIL-003	semantic	100%	receive TLS 1.2 or prior ClientHellos which contain other compression methods and (if negotiating such a prior version) MUST follow the procedures for the appropriate prior version of TLS.	it sent in the ClientHello MUST abort the handshake with an
85	FAIL-003	semantic	100%	HRR (HelloRetryRequest). If an implementation receives an extension which it recognizes and which is not specified for the message in which it appears, it MUST abort the handshake with an	Implementations of this specification MUST send this extension in the
86	FAIL-003	semantic	100%	The length of the Salt MUST be equal to the length of the output	it MUST use the RSASSA-PSS OID [RFC5756]. When used in certificate signatures, the algorithm parameters MUST be DER
87	FAIL-003	semantic	100%	The server will send this message in response to a ClientHello	but the remaining bytes MUST be random. This structure is generated by the server and MUST be generated independently of the
88	FAIL-003	semantic	100%	and which receives a TLS 1.3 ServerHello during renegotiation MUST	process the extensions, starting with determining the version using "supported_versions". Clients MUST abort the handshake with an
89	FAIL-003	semantic	100%	negotiation, then this will happen automatically). Upon receiving the ServerHello, clients MUST check that the cipher suite supplied in	receiving such an extension, an endpoint MUST abort the handshake
90	FAIL-003	semantic	100%	validate using any signature algorithm using an MD5 hash MUST abort	up to this point. Servers MUST send this message when authenticating via a certificate. Clients MUST send this message whenever
91	FAIL-003	semantic	100%	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the	server to authenticate itself via a certificate MUST send the
92	FAIL-003	semantic	100%	this field MUST be non-empty, so a client not offering a pre-TLS 1.3 session MUST generate a new 32-byte value. This value	a server MUST check that the message either contains no data after
93	FAIL-003	semantic	100%	If the server supplies an empty Certificate message, the client MUST abort the handshake with a "decode_error" alert.	up to this point. Servers MUST send this message when authenticating via a certificate. Clients MUST send this message whenever
94	FAIL-003	semantic	100%	The server MUST send a Certificate message whenever the agreed-upon	field having length 0). A Finished message MUST be sent regardless
95	FAIL-003	semantic	100%	up to this point. Servers MUST send this message when authenticating via a certificate. Clients MUST send this message whenever	If the verification fails, the receiver MUST terminate the handshake
96	FAIL-003	semantic	100%	support TLS 1.3 will send a normal TLS 1.3 ClientHello containing	"supported_versions" extension is not present, the server MUST negotiate the minimum of ClientHello.legacy_version and TLS 1.2. For
97	FAIL-003	semantic	100%	Any ticket MUST only be resumed with a cipher suite that has the same	that reject early data due to lack of cryptographic material will
98	FAIL-003	semantic	100%	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.	The length of the Salt MUST be equal to the length of the digest
99	FAIL-003	semantic	100%	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.	The length of the Salt MUST be equal to the length of the output
100	FAIL-003	semantic	100%	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the	TLS 1.3, the client MUST abort the handshake with an

101	FAIL-003	semantic	100%	must still be handled at the TLS layer, though. Later documents may	alert has been received MUST be ignored. If a transport-level close
102	FAIL-003	semantic	100%	implementation MUST send alert X if it sends any alert. All alerts	message not containing an extension that is mandatory to send for
103	FAIL-003	semantic	100%	messages for a given response MUST appear consecutively on the wire	servers MUST be prepared for some delay, including receiving an
104	FAIL-003	semantic	100%	A server which negotiates a version of TLS prior to TLS 1.3 MUST set	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the
105	FAIL-003	semantic	99%	OIDs. If the client ignored some of the required certificate	extension to the client. Clients MUST NOT act upon any information
106	FAIL-003	semantic	99%	cipher suite that was not offered MUST abort the handshake with an	response to a ClientHello MUST set the last 8 bytes of their Random
107	FAIL-003	semantic	99%	The client and the server must share knowledge that the connection is	close_notify: This alert notifies the recipient that the sender will not send any more messages on this connection. Any data received after a closure alert has been received MUST be ignored.
108	FAIL-003	semantic	99%	ClientHello). When sending the new ClientHello, the client MUST copy	it MUST use the RSASSA-PSS OID [RFC5756]. When used in certificate signatures, the algorithm parameters MUST be DER
109	FAIL-003	semantic	99%	alert has been received MUST be ignored. If a transport-level close	implementation MUST send alert X if it sends any alert. All alerts
110	FAIL-003	semantic	99%	up to this point. Servers MUST send this message when authenticating via a certificate. Clients MUST send this message whenever	The receiver of a CertificateVerify message MUST verify the signature
111	FAIL-003	semantic	99%	process the extensions, starting with determining the version using "supported_versions". Clients MUST abort the handshake with an	was itself in response to a HelloRetryRequest), it MUST abort the handshake with an "unexpected_message" alert.
112	FAIL-003	semantic	99%	"supported_versions" extension is not present, the server MUST negotiate the minimum of ClientHello.legacy_version and TLS 1.2. For	change_cipher_spec at any time during the handshake, as they must be
113	FAIL-003	semantic	99%	appear at a point at the handshake where the implementation is	- Handshake messages MUST NOT be interleaved with other record
114	FAIL-003	semantic	99%	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.	server to authenticate itself via a certificate MUST send the
115	FAIL-003	semantic	99%	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the	extension, then the server MUST abort the handshake with a
116	FAIL-003	semantic	99%	The length of the Salt MUST be equal to the length of the digest	it MUST use the RSASSA-PSS OID [RFC5756]. When used in certificate signatures, the algorithm parameters MUST be DER
117	FAIL-003	semantic	99%	it MUST use the RSASSA-PSS OID [RFC5756]. When used in certificate signatures, the algorithm parameters MUST be DER	offering these values MUST list them as the lowest priority
118	FAIL-003	semantic	99%	which is different from an existing record which will be accepted	to cause the receiver to accept a record which it has already
119	FAIL-003	semantic	99%	it MUST use the RSASSA-PSS OID [RFC5756]. When used in certificate signatures, the algorithm parameters MUST be DER	mandatory in TLS 1.3) MUST be prepared to accept a signature using
120	FAIL-003	semantic	99%	a server MUST check that the message either contains no data after	cipher suite that was not offered MUST abort the handshake with an
121	FAIL-003	semantic	99%	the client MUST send an EndOfEarlyData message after receiving the	implementations MUST use the value sent in the resumption ClientHello
122	FAIL-003	semantic	99%	TLS 1.3, the client MUST abort the handshake with an	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.
123	FAIL-003	semantic	99%	then each side will also send a response, with the result that each	- Handshake messages MUST NOT be interleaved with other record

124	FAIL-003	semantic	99%	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.
125	FAIL-003	semantic	99%	Servers MUST check that it is the last extension and otherwise fail	certificates. The client MUST check EncryptedExtensions for the presence of any forbidden extensions and if any are found MUST abort
126	FAIL-003	semantic	99%	server to authenticate itself via a certificate MUST send the	extension, then the server MUST abort the handshake with a
127	FAIL-003	semantic	99%	The receiver of a CertificateVerify message MUST verify the signature	Any records following a Finished message MUST be encrypted under the
128	FAIL-003	semantic	99%	receives any other value, it MUST terminate the connection with an	receiver MUST send a KeyUpdate of its own with request_update set to
129	FAIL-003	semantic	99%	not support TLS 1.3, it will respond with a ServerHello containing an	(or is not acceptable), the client MUST abort the handshake with a
130	FAIL-003	semantic	99%	receive TLS 1.2 or prior ClientHellos which contain other compression methods and (if negotiating such a prior version) MUST follow the procedures for the appropriate prior version of TLS.	cipher suite that was not offered MUST abort the handshake with an
131	FAIL-003	semantic	99%	If the server successfully selects parameters and does not require a	- When authenticating via a certificate, the server will send the
132	FAIL-003	semantic	99%	AlertLevel=fatal and MUST be treated as error alerts when received	alert has been received MUST be ignored. If a transport-level close
133	FAIL-003	semantic	99%	a server MUST check that the message either contains no data after	it sent in the ClientHello MUST abort the handshake with an
134	FAIL-003	semantic	99%	message, a different PSK will be derived for each ticket.	and Finished. If it declines, it MUST send a Certificate message
135	FAIL-003	semantic	99%	server MUST abort the handshake with a "handshake_failure" or an	If the server successfully selects parameters and does not require a
136	FAIL-003	semantic	99%	Upon receipt of a HelloRetryRequest, the client MUST check the	receiving such an extension, an endpoint MUST abort the handshake
137	FAIL-003	semantic	99%	curve. If TLS 1.2 is negotiated, implementations MUST be prepared to accept a signature that uses any curve that they advertised in	mandatory in TLS 1.3) MUST be prepared to accept a signature using
138	FAIL-003	semantic	99%	(or is not acceptable), the client MUST abort the handshake with a	legacy_version is TLS 1.0, the server will proceed with a TLS 1.0
139	FAIL-003	semantic	99%	- The certificate MUST allow the key to be used for signing (i.e., the digitalSignature bit MUST be set if the Key Usage extension is	certificates and decides to abort the handshake, then it MUST abort
140	FAIL-003	semantic	99%	well. Extensions in the Certificate message from the server MUST correspond to ones from the ClientHello message. Extensions in the Certificate message from the client MUST correspond to	response, the body of its "status_request" extension MUST be a
141	FAIL-003	semantic	99%	multiple zones where tickets from zone A will not be accepted in	guarantee that the 0-RTT data is accepted at most once. Servers
142	FAIL-003	semantic	99%	- If containing a "supported_groups" extension, it MUST also contain	- A server receiving a ClientHello MUST correctly ignore all
143	FAIL-003	semantic	99%	A server which negotiates a version of TLS prior to TLS 1.3 MUST set	TLS 1.3, the client MUST abort the handshake with an
144	FAIL-003	semantic	99%	find a non-zero octet in the cleartext, it MUST terminate the	alert has been received MUST be ignored. If a transport-level close
145	FAIL-003	semantic	98%	TLS to first hash the input with a specified hash algorithm and then proceed as usual. The code point groups listed above have the	it MUST use the RSASSA-PSS OID [RFC5756]. When used in certificate signatures, the algorithm parameters MUST be DER
146	FAIL-003	semantic	98%	The length of the Salt MUST be equal to the length of the digest	curve. If TLS 1.2 is negotiated, implementations MUST be prepared to accept a signature that uses any curve that they advertised in

147	FAIL-003	semantic	98%	field having length 0). A Finished message MUST be sent regardless	well. Extensions in the Certificate message from the server MUST correspond to ones from the ClientHello message. Extensions in the Certificate message from the client MUST correspond to
148	FAIL-003	semantic	98%	and Finished. If it declines, it MUST send a Certificate message	and before the peer's Finished message has been received and MUST
149	FAIL-003	semantic	98%	Upon receiving a message with type server_hello, implementations MUST first examine the Random value and, if it matches this value, process	MUST check that the last 8 bytes are not equal to either of these
150	FAIL-003	semantic	98%	HRR (HelloRetryRequest). If an implementation receives an extension which it recognizes and which is not specified for the message in which it appears, it MUST abort the handshake with an	versions of TLS SHOULD support TLS 1.2. Servers MUST be prepared to receive ClientHellos that include this extension but do not include
151	FAIL-003	semantic	98%	enforces all-zero padding octets, which allows for quick detection of	find a non-zero octet in the cleartext, it MUST terminate the
152	FAIL-003	semantic	98%	this field MUST be non-empty, so a client not offering a pre-TLS 1.3 session MUST generate a new 32-byte value. This value	block with no data following. If not, then it MUST abort the handshake with a "decode_error" alert.
153	FAIL-003	semantic	98%	PSK the server ultimately selects. The client will need to compute one for each potential PSK; if no PSK is selected, it will then need to compute the Early Secret corresponding to the zero PSK.	deployments will maintain state at that level. Therefore, in normal operation, clients will not know which, if any, of these mechanisms servers actually implement and hence MUST only send early data which
154	FAIL-003	semantic	98%	handshake when the server does not have any information about the	identity will result in a failed handshake, whereas an invalid identity will just be skipped and result in a successful certificate
155	FAIL-003	semantic	98%	"supported_versions" extension is not present, the server MUST negotiate the minimum of ClientHello.legacy_version and TLS 1.2. For	legacy_version is TLS 1.0, the server will proceed with a TLS 1.0
156	FAIL-003	semantic	98%	request a certificate from the client. This message, if sent, MUST follow EncryptedExtensions.	The server MUST send a Certificate message whenever the agreed-upon
157	FAIL-003	semantic	98%	Any records following a Finished message MUST be encrypted under the	transmitted by the client. The server MUST generate a fresh value
158	FAIL-003	semantic	98%	prevent an attacker who has temporary access to the client's	well. Extensions in the Certificate message from the server MUST correspond to ones from the ClientHello message. Extensions in the Certificate message from the client MUST correspond to
159	FAIL-003	semantic	97%	If the server supplies an empty Certificate message, the client MUST abort the handshake with a "decode_error" alert.	is non-empty). When sent, this message MUST appear immediately after
160	FAIL-003	semantic	97%	find a non-zero octet in the cleartext, it MUST terminate the	close_notify: This alert notifies the recipient that the sender will not send any more messages on this connection. Any data received after a closure alert has been received MUST be ignored.
161	FAIL-003	semantic	97%	TLS 1.3, the client MUST abort the handshake with an	server to authenticate itself via a certificate MUST send the
162	FAIL-003	semantic	97%	Upon receipt of a HelloRetryRequest, the client MUST check the	negotiation, then this will happen automatically). Upon receiving the ServerHello, clients MUST check that the cipher suite supplied in
163	FAIL-003	semantic	97%	also implement data stores with false positives, such as Bloom filters, in which case they MUST respond to apparent replay by rejecting 0-RTT but MUST NOT abort the handshake.	zone will accept 0-RTT data. A weaker design is to implement
164	FAIL-003	semantic	97%	clients MUST forget the secret values and keys established in failed	Each party MUST send a "close_notify" alert before closing its write
165	FAIL-003	semantic	97%	The server MUST send a Certificate message whenever the agreed-upon	well. Extensions in the Certificate message from the server MUST correspond to ones from the ClientHello message. Extensions in the Certificate message from the client MUST correspond to

166	FAIL-003	semantic	97%	- If not containing a "pre_shared_key" extension, it MUST contain	- A server receiving a ClientHello MUST correctly ignore all
167	FAIL-003	semantic	97%	MUST check that the last 8 bytes are not equal to either of these	process the extensions, starting with determining the version using "supported_versions". Clients MUST abort the handshake with an
168	FAIL-003	semantic	97%	but the remaining bytes MUST be random. This structure is generated by the server and MUST be generated independently of the	If negotiating TLS 1.2, TLS 1.3 servers MUST set the last 8 bytes of
169	FAIL-003	semantic	96%	must still be handled at the TLS layer, though. Later documents may	clients MUST forget the secret values and keys established in failed
170	FAIL-003	semantic	96%	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.	it MUST use the RSASSA-PSS OID [RFC5756]. When used in certificate signatures, the algorithm parameters MUST be DER
171	FAIL-003	semantic	96%	still require consistent storage for anti-replay of 0-RTT data, as	the storage key, then this ClientHello will not appear as a duplicate, which will cause the ClientHello to be accepted, and may cause side effects such as replay cache pollution, although any 0-RTT data will not be decryptable because it will use different keys. If
172	FAIL-003	semantic	96%	0x0303 (TLS 1.2). Clients MUST check for this extension prior to processing the rest of the ServerHello (although they will have to	TLS 1.3, the client MUST abort the handshake with an
173	FAIL-003	semantic	96%	transmitted by the client. The server MUST generate a fresh value	and Finished. If it declines, it MUST send a Certificate message
174	FAIL-003	semantic	96%	Any records following a Finished message MUST be encrypted under the	implementations MUST use the value sent in the resumption ClientHello
175	FAIL-003	semantic	96%	guarantee that the 0-RTT data is accepted at most once. Servers	the storage key, then this ClientHello will not appear as a duplicate, which will cause the ClientHello to be accepted, and may cause side effects such as replay cache pollution, although any 0-RTT data will not be decryptable because it will use different keys. If
176	FAIL-003	semantic	96%	HelloRetryRequest. In that case, the client MUST send the same	TLS 1.3 and receives a ClientHello at any other time, it MUST terminate the connection with an "unexpected_message" alert.
177	FAIL-003	semantic	95%	transmitted by the client. The server MUST generate a fresh value	servers MUST be prepared for some delay, including receiving an
178	FAIL-003	semantic	95%	also implement data stores with false positives, such as Bloom filters, in which case they MUST respond to apparent replay by rejecting 0-RTT but MUST NOT abort the handshake.	the storage key, then this ClientHello will not appear as a duplicate, which will cause the ClientHello to be accepted, and may cause side effects such as replay cache pollution, although any 0-RTT data will not be decryptable because it will use different keys. If
179	FAIL-003	semantic	95%	of [RFC6066]), with the server being required to enforce that the SNI value associated with the session matches the one specified in the	Clients MUST verify that the server's selected_identity is within the
180	FAIL-003	semantic	94%	must still be handled at the TLS layer, though. Later documents may	close_notify: This alert notifies the recipient that the sender will not send any more messages on this connection. Any data received after a closure alert has been received MUST be ignored.
181	FAIL-003	semantic	94%	The length of the Salt MUST be equal to the length of the output	curve. If TLS 1.2 is negotiated, implementations MUST be prepared to accept a signature that uses any curve that they advertised in
182	FAIL-003	semantic	94%	Prior to accepting PSK key establishment, the server MUST validate	The "pre_shared_key" extension MUST be the last extension in the
183	FAIL-003	semantic	93%	and Finished. If it declines, it MUST send a Certificate message	message MUST terminate the connection with an "unexpected_message" alert. After sending a KeyUpdate message, the sender SHALL send all
184	FAIL-003	semantic	93%	AlertLevel=fatal and MUST be treated as error alerts when received	close_notify: This alert notifies the recipient that the sender will not send any more messages on this connection. Any data received after a closure alert has been received MUST be ignored.

185	FAIL-003	semantic	93%	MUST be specified, and other extensions may optionally be included	well. Extensions in the Certificate message from the server MUST correspond to ones from the ClientHello message. Extensions in the Certificate message from the client MUST correspond to
186	FAIL-003	semantic	93%	present, then the parameters in the signature MUST be identical to	offering these values MUST list them as the lowest priority
187	FAIL-003	semantic	93%	- A client sending a ClientHello MUST support all parameters	compliant TLS client (to the original server), including verifying the original server's certificate. In particular, it MUST generate its own ClientHello containing only parameters it understands, and it MUST generate a fresh ServerHello random
188	FAIL-003	semantic	93%	support TLS 1.3 will send a normal TLS 1.3 ClientHello containing	A client that attempts to send 0-RTT data MUST fail a connection if
189	FAIL-003	semantic	93%	to cause the receiver to accept a record which it has already	(which processes the data immediately) and another cluster which will fall back to 1-RTT and process the data upon application-layer
190	FAIL-003	semantic	93%	TLS 1.3, the client MUST abort the handshake with an	ClientHello). When sending the new ClientHello, the client MUST copy
191	FAIL-003	semantic	93%	Upon receiving a message with type server_hello, implementations MUST first examine the Random value and, if it matches this value, process	below. If a match is found, the client MUST abort the handshake with
192	FAIL-003	semantic	92%	Upon receipt of a HelloRetryRequest, the client MUST check the	"supported_versions" extension MUST be retained in the ServerHello, and a client MUST abort the handshake with an "illegal_parameter"
193	FAIL-003	semantic	92%	receiving such an extension, an endpoint MUST abort the handshake	versions of TLS SHOULD support TLS 1.2. Servers MUST be prepared to receive ClientHellos that include this extension but do not include
194	FAIL-003	semantic	92%	cipher suite that was not offered MUST abort the handshake with an	and which receives a TLS 1.3 ServerHello during renegotiation MUST
195	FAIL-003	semantic	91%	TLS 1.x versions in this field, but its value MUST always be ignored.	change_cipher_spec at any time during the handshake, as they must be
196	FAIL-004	modality	90%	to this document. Code Components extracted from this document must	The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT",
197	FAIL-004	modality	90%	The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT",	the server's ephemeral Diffie-Hellman share; the server's share MUST be in the same group as one of the client's shares. If PSK key
198	FAIL-004	modality	90%	The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT",	Application Data MUST NOT be sent prior to sending the Finished
199	FAIL-004	modality	90%	The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT",	common cryptographic parameters can be negotiated, the server MUST abort the handshake with an appropriate alert.
200	FAIL-004	modality	90%	the server's ephemeral Diffie-Hellman share; the server's share MUST be in the same group as one of the client's shares. If PSK key	Application Data MUST NOT be sent prior to sending the Finished
201	FAIL-004	modality	90%	Application Data MUST NOT be sent prior to sending the Finished	common cryptographic parameters can be negotiated, the server MUST abort the handshake with an appropriate alert.
202	FAIL-004	modality	90%	Application Data MUST NOT be sent prior to sending the Finished	hash algorithm to be used with the PSK MUST also be provisioned.
203	FAIL-004	modality	90%	it MUST abort the handshake with either a "handshake_failure" or	TLS and receives a TLS 1.3 ClientHello in a renegotiation, it MUST retain the previous protocol version. In particular, it MUST NOT negotiate TLS 1.3.
204	FAIL-004	modality	90%	HelloRetryRequest. In that case, the client MUST send the same	TLS and receives a TLS 1.3 ClientHello in a renegotiation, it MUST retain the previous protocol version. In particular, it MUST NOT negotiate TLS 1.3.

205	FAIL-004	modality	90%	TLS 1.3 and receives a ClientHello at any other time, it MUST terminate the connection with an "unexpected_message" alert.	TLS and receives a TLS 1.3 ClientHello in a renegotiation, it MUST retain the previous protocol version. In particular, it MUST NOT negotiate TLS 1.3.
206	FAIL-004	modality	90%	TLS and receives a TLS 1.3 ClientHello in a renegotiation, it MUST retain the previous protocol version. In particular, it MUST NOT negotiate TLS 1.3.	legacy_version field MUST be set to 0x0303, which is the version
207	FAIL-004	modality	90%	TLS and receives a TLS 1.3 ClientHello in a renegotiation, it MUST retain the previous protocol version. In particular, it MUST NOT negotiate TLS 1.3.	this field MUST be non-empty, so a client not offering a pre-TLS 1.3 session MUST generate a new 32-byte value. This value
208	FAIL-004	modality	90%	TLS and receives a TLS 1.3 ClientHello in a renegotiation, it MUST retain the previous protocol version. In particular, it MUST NOT negotiate TLS 1.3.	ossification). Otherwise, it MUST be set as a zero-length vector
209	FAIL-004	modality	90%	TLS and receives a TLS 1.3 ClientHello in a renegotiation, it MUST retain the previous protocol version. In particular, it MUST NOT negotiate TLS 1.3.	does not recognize, support, or wish to use, the server MUST ignore those cipher suites and process the remaining ones as
210	FAIL-004	modality	90%	TLS and receives a TLS 1.3 ClientHello in a renegotiation, it MUST retain the previous protocol version. In particular, it MUST NOT negotiate TLS 1.3.	MUST contain exactly one byte, set to zero, which corresponds to
211	FAIL-004	modality	90%	TLS and receives a TLS 1.3 ClientHello in a renegotiation, it MUST retain the previous protocol version. In particular, it MUST NOT negotiate TLS 1.3.	field, the server MUST abort the handshake with an
212	FAIL-004	modality	90%	TLS and receives a TLS 1.3 ClientHello in a renegotiation, it MUST retain the previous protocol version. In particular, it MUST NOT negotiate TLS 1.3.	receive TLS 1.2 or prior ClientHellos which contain other compression methods and (if negotiating such a prior version) MUST follow the procedures for the appropriate prior version of TLS.
213	FAIL-004	modality	90%	TLS and receives a TLS 1.3 ClientHello in a renegotiation, it MUST retain the previous protocol version. In particular, it MUST NOT negotiate TLS 1.3.	versions of TLS. Servers MUST ignore unrecognized extensions.
214	FAIL-004	modality	90%	extensions: A list of extensions. The ServerHello MUST only include extensions which are required to establish the cryptographic	with the ServerHello, a HelloRetryRequest MUST NOT contain any
215	FAIL-004	modality	90%	Upon receiving a message with type server_hello, implementations MUST first examine the Random value and, if it matches this value, process	with the ServerHello, a HelloRetryRequest MUST NOT contain any
216	FAIL-004	modality	90%	response to a ClientHello MUST set the last 8 bytes of their Random	with the ServerHello, a HelloRetryRequest MUST NOT contain any
217	FAIL-004	modality	90%	If negotiating TLS 1.2, TLS 1.3 servers MUST set the last 8 bytes of	with the ServerHello, a HelloRetryRequest MUST NOT contain any
218	FAIL-004	modality	90%	If negotiating TLS 1.1 or below, TLS 1.3 servers MUST, and TLS 1.2	with the ServerHello, a HelloRetryRequest MUST NOT contain any
219	FAIL-004	modality	90%	MUST check that the last 8 bytes are not equal to either of these	with the ServerHello, a HelloRetryRequest MUST NOT contain any
220	FAIL-004	modality	90%	below. If a match is found, the client MUST abort the handshake with	with the ServerHello, a HelloRetryRequest MUST NOT contain any
221	FAIL-004	modality	90%	and which receives a TLS 1.3 ServerHello during renegotiation MUST	with the ServerHello, a HelloRetryRequest MUST NOT contain any
222	FAIL-004	modality	90%	and which receives a TLS 1.3 ServerHello during renegotiation MUST	Implementations MUST NOT send extension responses if the remote
223	FAIL-004	modality	90%	with the ServerHello, a HelloRetryRequest MUST NOT contain any	Upon receipt of a HelloRetryRequest, the client MUST check the

224	FAIL-004	modality	90%	with the ServerHello, a HelloRetryRequest MUST NOT contain any	process the extensions, starting with determining the version using "supported_versions". Clients MUST abort the handshake with an
225	FAIL-004	modality	90%	with the ServerHello, a HelloRetryRequest MUST NOT contain any	was itself in response to a HelloRetryRequest), it MUST abort the handshake with an "unexpected_message" alert.
226	FAIL-004	modality	90%	with the ServerHello, a HelloRetryRequest MUST NOT contain any	Otherwise, the client MUST process all extensions in the
227	FAIL-004	modality	90%	with the ServerHello, a HelloRetryRequest MUST NOT contain any	A client which receives a cipher suite that was not offered MUST abort the handshake. Servers MUST ensure that they negotiate the
228	FAIL-004	modality	90%	with the ServerHello, a HelloRetryRequest MUST NOT contain any	negotiation, then this will happen automatically). Upon receiving the ServerHello, clients MUST check that the cipher suite supplied in
229	FAIL-004	modality	90%	with the ServerHello, a HelloRetryRequest MUST NOT contain any	"supported_versions" extension MUST be retained in the ServerHello, and a client MUST abort the handshake with an "illegal_parameter"
230	FAIL-004	modality	90%	with the ServerHello, a HelloRetryRequest MUST NOT contain any	Implementations MUST NOT send extension responses if the remote
231	FAIL-004	modality	90%	with the ServerHello, a HelloRetryRequest MUST NOT contain any	receiving such an extension, an endpoint MUST abort the handshake
232	FAIL-004	modality	90%	with the ServerHello, a HelloRetryRequest MUST NOT contain any	HRR (HelloRetryRequest). If an implementation receives an extension which it recognizes and which is not specified for the message in which it appears, it MUST abort the handshake with an
233	FAIL-004	modality	90%	Upon receipt of a HelloRetryRequest, the client MUST check the	Implementations MUST NOT send extension responses if the remote
234	FAIL-004	modality	90%	Upon receipt of a HelloRetryRequest, the client MUST check the	extensions block). There MUST NOT be more than one extension of the
235	FAIL-004	modality	90%	process the extensions, starting with determining the version using "supported_versions". Clients MUST abort the handshake with an	Implementations MUST NOT send extension responses if the remote
236	FAIL-004	modality	90%	process the extensions, starting with determining the version using "supported_versions". Clients MUST abort the handshake with an	extensions block). There MUST NOT be more than one extension of the
237	FAIL-004	modality	90%	was itself in response to a HelloRetryRequest), it MUST abort the handshake with an "unexpected_message" alert.	Implementations MUST NOT send extension responses if the remote
238	FAIL-004	modality	90%	was itself in response to a HelloRetryRequest), it MUST abort the handshake with an "unexpected_message" alert.	extensions block). There MUST NOT be more than one extension of the
239	FAIL-004	modality	90%	Otherwise, the client MUST process all extensions in the	Implementations MUST NOT send extension responses if the remote
240	FAIL-004	modality	90%	Otherwise, the client MUST process all extensions in the	extensions block). There MUST NOT be more than one extension of the
241	FAIL-004	modality	90%	A client which receives a cipher suite that was not offered MUST abort the handshake. Servers MUST ensure that they negotiate the	Implementations MUST NOT send extension responses if the remote
242	FAIL-004	modality	90%	A client which receives a cipher suite that was not offered MUST abort the handshake. Servers MUST ensure that they negotiate the	extensions block). There MUST NOT be more than one extension of the
243	FAIL-004	modality	90%	negotiation, then this will happen automatically). Upon receiving the ServerHello, clients MUST check that the cipher suite supplied in	Implementations MUST NOT send extension responses if the remote
244	FAIL-004	modality	90%	negotiation, then this will happen automatically). Upon receiving the ServerHello, clients MUST check that the cipher suite supplied in	extensions block). There MUST NOT be more than one extension of the
245	FAIL-004	modality	90%	"supported_versions" extension MUST be retained in the ServerHello, and a client MUST abort the handshake with an "illegal_parameter"	Implementations MUST NOT send extension responses if the remote

246	FAIL-004	modality	90%	"supported_versions" extension MUST be retained in the ServerHello, and a client MUST abort the handshake with an "illegal_parameter"	extensions block). There MUST NOT be more than one extension of the
247	FAIL-004	modality	90%	"supported_versions" extension MUST be retained in the ServerHello, and a client MUST abort the handshake with an "illegal_parameter"	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present
248	FAIL-004	modality	90%	Implementations MUST NOT send extension responses if the remote	receiving such an extension, an endpoint MUST abort the handshake
249	FAIL-004	modality	90%	Implementations MUST NOT send extension responses if the remote	HRR (HelloRetryRequest). If an implementation receives an extension which it recognizes and which is not specified for the message in which it appears, it MUST abort the handshake with an
250	FAIL-004	modality	90%	Implementations MUST NOT send extension responses if the remote	extensions block). There MUST NOT be more than one extension of the
251	FAIL-004	modality	90%	Implementations MUST NOT send extension responses if the remote	Implementations of this specification MUST send this extension in the
252	FAIL-004	modality	90%	Implementations MUST NOT send extension responses if the remote	if previous versions of TLS are allowed to be negotiated, they MUST be present as well).
253	FAIL-004	modality	90%	Implementations MUST NOT send extension responses if the remote	this specification and which also support TLS 1.2 MUST negotiate
254	FAIL-004	modality	90%	Implementations MUST NOT send extension responses if the remote	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present
255	FAIL-004	modality	90%	Implementations MUST NOT send extension responses if the remote	versions of TLS SHOULD support TLS 1.2. Servers MUST be prepared to receive ClientHellos that include this extension but do not include
256	FAIL-004	modality	90%	receiving such an extension, an endpoint MUST abort the handshake	extensions block). There MUST NOT be more than one extension of the
257	FAIL-004	modality	90%	receiving such an extension, an endpoint MUST abort the handshake	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present
258	FAIL-004	modality	90%	HRR (HelloRetryRequest). If an implementation receives an extension which it recognizes and which is not specified for the message in which it appears, it MUST abort the handshake with an	extensions block). There MUST NOT be more than one extension of the
259	FAIL-004	modality	90%	HRR (HelloRetryRequest). If an implementation receives an extension which it recognizes and which is not specified for the message in which it appears, it MUST abort the handshake with an	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present

260	FAIL-004	modality	90%	HRR (HelloRetryRequest). If an implementation receives an extension which it recognizes and which is not specified for the message in which it appears, it MUST abort the handshake with an	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending
261	FAIL-004	modality	90%	extensions block). There MUST NOT be more than one extension of the	Implementations of this specification MUST send this extension in the
262	FAIL-004	modality	90%	extensions block). There MUST NOT be more than one extension of the	if previous versions of TLS are allowed to be negotiated, they MUST be present as well).
263	FAIL-004	modality	90%	extensions block). There MUST NOT be more than one extension of the	this specification and which also support TLS 1.2 MUST negotiate
264	FAIL-004	modality	90%	extensions block). There MUST NOT be more than one extension of the	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present
265	FAIL-004	modality	90%	extensions block). There MUST NOT be more than one extension of the	versions of TLS SHOULD support TLS 1.2. Servers MUST be prepared to receive ClientHellos that include this extension but do not include
266	FAIL-004	modality	90%	extensions block). There MUST NOT be more than one extension of the	A server which negotiates a version of TLS prior to TLS 1.3 MUST set
267	FAIL-004	modality	90%	extensions block). There MUST NOT be more than one extension of the	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending
268	FAIL-004	modality	90%	extensions block). There MUST NOT be more than one extension of the	value (0x0304). It MUST set the ServerHello.legacy_version field to
269	FAIL-004	modality	90%	Implementations of this specification MUST send this extension in the	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present
270	FAIL-004	modality	90%	Implementations of this specification MUST send this extension in the	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending
271	FAIL-004	modality	90%	if previous versions of TLS are allowed to be negotiated, they MUST be present as well).	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present
272	FAIL-004	modality	90%	if previous versions of TLS are allowed to be negotiated, they MUST be present as well).	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending
273	FAIL-004	modality	90%	this specification and which also support TLS 1.2 MUST negotiate	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present

274	FAIL-004	modality	90%	this specification and which also support TLS 1.2 MUST negotiate	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending
275	FAIL-004	modality	90%	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present	versions of TLS SHOULD support TLS 1.2. Servers MUST be prepared to receive ClientHellos that include this extension but do not include
276	FAIL-004	modality	90%	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present	A server which negotiates a version of TLS prior to TLS 1.3 MUST set
277	FAIL-004	modality	90%	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending
278	FAIL-004	modality	90%	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present	value (0x0304). It MUST set the ServerHello.legacy_version field to
279	FAIL-004	modality	90%	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present	0x0303 (TLS 1.2). Clients MUST check for this extension prior to processing the rest of the ServerHello (although they will have to
280	FAIL-004	modality	90%	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the
281	FAIL-004	modality	90%	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present	TLS 1.3, the client MUST abort the handshake with an

282	FAIL-004	modality	90%	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present	ClientHello). When sending the new ClientHello, the client MUST copy
283	FAIL-004	modality	90%	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present	a "cookie" extension in the new ClientHello. Clients MUST NOT use
284	FAIL-004	modality	90%	If this extension is present in the ClientHello, servers MUST NOT use the ClientHello.legacy_version value for version negotiation and MUST use only the "supported_versions" extension to determine client preferences. Servers MUST only select a version of TLS present in that extension and MUST ignore any unknown versions that are present	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.
285	FAIL-004	modality	90%	versions of TLS SHOULD support TLS 1.2. Servers MUST be prepared to receive ClientHellos that include this extension but do not include	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending
286	FAIL-004	modality	90%	versions of TLS SHOULD support TLS 1.2. Servers MUST be prepared to receive ClientHellos that include this extension but do not include	a "cookie" extension in the new ClientHello. Clients MUST NOT use
287	FAIL-004	modality	90%	A server which negotiates a version of TLS prior to TLS 1.3 MUST set	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending
288	FAIL-004	modality	90%	A server which negotiates a version of TLS prior to TLS 1.3 MUST set	a "cookie" extension in the new ClientHello. Clients MUST NOT use
289	FAIL-004	modality	90%	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending	value (0x0304). It MUST set the ServerHello.legacy_version field to
290	FAIL-004	modality	90%	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending	0x0303 (TLS 1.2). Clients MUST check for this extension prior to processing the rest of the ServerHello (although they will have to
291	FAIL-004	modality	90%	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the
292	FAIL-004	modality	90%	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending	TLS 1.3, the client MUST abort the handshake with an
293	FAIL-004	modality	90%	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending	ClientHello). When sending the new ClientHello, the client MUST copy
294	FAIL-004	modality	90%	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending	a "cookie" extension in the new ClientHello. Clients MUST NOT use
295	FAIL-004	modality	90%	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.
296	FAIL-004	modality	90%	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending	CertificateVerify messages. The keys found in certificates MUST also

297	FAIL-004	modality	90%	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending	server to authenticate itself via a certificate MUST send the
298	FAIL-004	modality	90%	ServerHello.version and MUST NOT send the "supported_versions" extension. A server which negotiates TLS 1.3 MUST respond by sending	extension, then the server MUST abort the handshake with a
299	FAIL-004	modality	90%	value (0x0304). It MUST set the ServerHello.legacy_version field to	a "cookie" extension in the new ClientHello. Clients MUST NOT use
300	FAIL-004	modality	90%	0x0303 (TLS 1.2). Clients MUST check for this extension prior to processing the rest of the ServerHello (although they will have to	a "cookie" extension in the new ClientHello. Clients MUST NOT use
301	FAIL-004	modality	90%	parse the ServerHello in order to read the extension). If this extension is present, clients MUST ignore the	a "cookie" extension in the new ClientHello. Clients MUST NOT use
302	FAIL-004	modality	90%	TLS 1.3, the client MUST abort the handshake with an	a "cookie" extension in the new ClientHello. Clients MUST NOT use
303	FAIL-004	modality	90%	ClientHello). When sending the new ClientHello, the client MUST copy	a "cookie" extension in the new ClientHello. Clients MUST NOT use
304	FAIL-004	modality	90%	a "cookie" extension in the new ClientHello. Clients MUST NOT use	state, this will appear as if it were the first message to be received. Servers operating statelessly MUST ignore these records.
305	FAIL-004	modality	90%	a "cookie" extension in the new ClientHello. Clients MUST NOT use	CertificateVerify messages. The keys found in certificates MUST also
306	FAIL-004	modality	90%	a "cookie" extension in the new ClientHello. Clients MUST NOT use	server to authenticate itself via a certificate MUST send the
307	FAIL-004	modality	90%	a "cookie" extension in the new ClientHello. Clients MUST NOT use	extension, then the server MUST abort the handshake with a
308	FAIL-004	modality	90%	a "cookie" extension in the new ClientHello. Clients MUST NOT use	The length of the Salt MUST be equal to the length of the output
309	FAIL-004	modality	90%	a "cookie" extension in the new ClientHello. Clients MUST NOT use	certificate, it MUST use the rsaEncryption OID [RFC5280].
310	FAIL-004	modality	90%	a "cookie" extension in the new ClientHello. Clients MUST NOT use	The length of the Salt MUST be equal to the length of the digest
311	FAIL-004	modality	90%	a "cookie" extension in the new ClientHello. Clients MUST NOT use	it MUST use the RSASSA-PSS OID [RFC5756]. When used in certificate signatures, the algorithm parameters MUST be DER
312	FAIL-004	modality	90%	a "cookie" extension in the new ClientHello. Clients MUST NOT use	present, then the parameters in the signature MUST be identical to
313	FAIL-004	modality	90%	CertificateVerify messages. The keys found in certificates MUST also	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless
314	FAIL-004	modality	90%	server to authenticate itself via a certificate MUST send the	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless
315	FAIL-004	modality	90%	extension, then the server MUST abort the handshake with a	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless
316	FAIL-004	modality	90%	extension, then the server MUST abort the handshake with a	TLS 1.3. They MUST NOT be offered or negotiated by any
317	FAIL-004	modality	90%	The length of the Salt MUST be equal to the length of the output	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless
318	FAIL-004	modality	90%	The length of the Salt MUST be equal to the length of the output	TLS 1.3. They MUST NOT be offered or negotiated by any
319	FAIL-004	modality	90%	certificate, it MUST use the rsaEncryption OID [RFC5280].	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless
320	FAIL-004	modality	90%	certificate, it MUST use the rsaEncryption OID [RFC5280].	TLS 1.3. They MUST NOT be offered or negotiated by any
321	FAIL-004	modality	90%	The length of the Salt MUST be equal to the length of the digest	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless

322	FAIL-004	modality	90%	The length of the Salt MUST be equal to the length of the digest	TLS 1.3. They MUST NOT be offered or negotiated by any
323	FAIL-004	modality	90%	it MUST use the RSASSA-PSS OID [RFC5756]. When used in certificate signatures, the algorithm parameters MUST be DER	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless
324	FAIL-004	modality	90%	it MUST use the RSASSA-PSS OID [RFC5756]. When used in certificate signatures, the algorithm parameters MUST be DER	TLS 1.3. They MUST NOT be offered or negotiated by any
325	FAIL-004	modality	90%	present, then the parameters in the signature MUST be identical to	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless
326	FAIL-004	modality	90%	present, then the parameters in the signature MUST be identical to	TLS 1.3. They MUST NOT be offered or negotiated by any
327	FAIL-004	modality	90%	offering these values MUST list them as the lowest priority	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless
328	FAIL-004	modality	90%	offering these values MUST list them as the lowest priority	TLS 1.3. They MUST NOT be offered or negotiated by any
329	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless	implementations willing to negotiate TLS 1.2 MUST behave in
330	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless	TLS 1.3. They MUST NOT be offered or negotiated by any
331	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless	curve. If TLS 1.2 is negotiated, implementations MUST be prepared to accept a signature that uses any curve that they advertised in
332	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless	mandatory in TLS 1.3) MUST be prepared to accept a signature using
333	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless	match. This extension, if provided by the server, MUST only be sent
334	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless	filters list, the client certificate included in the response MUST contain all of the specified extension OIDs that the client
335	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless	of the specified values MUST be present in the client certificate
336	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT offer a SHA-1 signed certificate unless	client MUST ignore and skip any unrecognized certificate extension
337	FAIL-004	modality	90%	implementations willing to negotiate TLS 1.2 MUST behave in	TLS 1.3. They MUST NOT be offered or negotiated by any
338	FAIL-004	modality	90%	implementations willing to negotiate TLS 1.2 MUST behave in	with an "unsupported_certificate" alert. Any given OID MUST NOT appear more than once in the filters list.
339	FAIL-004	modality	90%	TLS 1.3. They MUST NOT be offered or negotiated by any	curve. If TLS 1.2 is negotiated, implementations MUST be prepared to accept a signature that uses any curve that they advertised in
340	FAIL-004	modality	90%	TLS 1.3. They MUST NOT be offered or negotiated by any	mandatory in TLS 1.3) MUST be prepared to accept a signature using
341	FAIL-004	modality	90%	TLS 1.3. They MUST NOT be offered or negotiated by any	match. This extension, if provided by the server, MUST only be sent
342	FAIL-004	modality	90%	TLS 1.3. They MUST NOT be offered or negotiated by any	filters list, the client certificate included in the response MUST contain all of the specified extension OIDs that the client
343	FAIL-004	modality	90%	TLS 1.3. They MUST NOT be offered or negotiated by any	of the specified values MUST be present in the client certificate
344	FAIL-004	modality	90%	TLS 1.3. They MUST NOT be offered or negotiated by any	client MUST ignore and skip any unrecognized certificate extension
345	FAIL-004	modality	90%	TLS 1.3. They MUST NOT be offered or negotiated by any	with an "unsupported_certificate" alert. Any given OID MUST NOT appear more than once in the filters list.

346	FAIL-004	modality	90%	curve. If TLS 1.2 is negotiated, implementations MUST be prepared to accept a signature that uses any curve that they advertised in	with an "unsupported_certificate" alert. Any given OID MUST NOT appear more than once in the filters list.
347	FAIL-004	modality	90%	curve. If TLS 1.2 is negotiated, implementations MUST be prepared to accept a signature that uses any curve that they advertised in	special anyExtendedKeyUsage OID MUST NOT be used in the request.
348	FAIL-004	modality	90%	mandatory in TLS 1.3) MUST be prepared to accept a signature using	with an "unsupported_certificate" alert. Any given OID MUST NOT appear more than once in the filters list.
349	FAIL-004	modality	90%	mandatory in TLS 1.3) MUST be prepared to accept a signature using	special anyExtendedKeyUsage OID MUST NOT be used in the request.
350	FAIL-004	modality	90%	mandatory in TLS 1.3) MUST be prepared to accept a signature using	Servers MUST NOT send a post-handshake CertificateRequest to clients which do not offer this extension. Servers MUST NOT send this
351	FAIL-004	modality	90%	match. This extension, if provided by the server, MUST only be sent	with an "unsupported_certificate" alert. Any given OID MUST NOT appear more than once in the filters list.
352	FAIL-004	modality	90%	match. This extension, if provided by the server, MUST only be sent	special anyExtendedKeyUsage OID MUST NOT be used in the request.
353	FAIL-004	modality	90%	match. This extension, if provided by the server, MUST only be sent	Servers MUST NOT send a post-handshake CertificateRequest to clients which do not offer this extension. Servers MUST NOT send this
354	FAIL-004	modality	90%	match. This extension, if provided by the server, MUST only be sent	extension to the client. Clients MUST NOT act upon any information
355	FAIL-004	modality	90%	filters list, the client certificate included in the response MUST contain all of the specified extension OIDs that the client	with an "unsupported_certificate" alert. Any given OID MUST NOT appear more than once in the filters list.
356	FAIL-004	modality	90%	filters list, the client certificate included in the response MUST contain all of the specified extension OIDs that the client	special anyExtendedKeyUsage OID MUST NOT be used in the request.
357	FAIL-004	modality	90%	filters list, the client certificate included in the response MUST contain all of the specified extension OIDs that the client	Servers MUST NOT send a post-handshake CertificateRequest to clients which do not offer this extension. Servers MUST NOT send this
358	FAIL-004	modality	90%	filters list, the client certificate included in the response MUST contain all of the specified extension OIDs that the client	extension to the client. Clients MUST NOT act upon any information
359	FAIL-004	modality	90%	of the specified values MUST be present in the client certificate	with an "unsupported_certificate" alert. Any given OID MUST NOT appear more than once in the filters list.
360	FAIL-004	modality	90%	of the specified values MUST be present in the client certificate	special anyExtendedKeyUsage OID MUST NOT be used in the request.
361	FAIL-004	modality	90%	of the specified values MUST be present in the client certificate	Servers MUST NOT send a post-handshake CertificateRequest to clients which do not offer this extension. Servers MUST NOT send this
362	FAIL-004	modality	90%	of the specified values MUST be present in the client certificate	extension to the client. Clients MUST NOT act upon any information
363	FAIL-004	modality	90%	client MUST ignore and skip any unrecognized certificate extension	with an "unsupported_certificate" alert. Any given OID MUST NOT appear more than once in the filters list.
364	FAIL-004	modality	90%	client MUST ignore and skip any unrecognized certificate extension	special anyExtendedKeyUsage OID MUST NOT be used in the request.
365	FAIL-004	modality	90%	client MUST ignore and skip any unrecognized certificate extension	Servers MUST NOT send a post-handshake CertificateRequest to clients which do not offer this extension. Servers MUST NOT send this
366	FAIL-004	modality	90%	client MUST ignore and skip any unrecognized certificate extension	extension to the client. Clients MUST NOT act upon any information

367	FAIL-004	modality	90%	client MUST ignore and skip any unrecognized certificate extension	key_exchange values for each KeyShareEntry MUST be generated independently. Clients MUST NOT offer multiple KeyShareEntry values for the same group. Clients MUST NOT offer any KeyShareEntry values
368	FAIL-004	modality	90%	with an "unsupported_certificate" alert. Any given OID MUST NOT appear more than once in the filters list.	special anyExtendedKeyUsage OID MUST NOT be used in the request.
369	FAIL-004	modality	90%	with an "unsupported_certificate" alert. Any given OID MUST NOT appear more than once in the filters list.	Servers MUST NOT send a post-handshake CertificateRequest to clients which do not offer this extension. Servers MUST NOT send this
370	FAIL-004	modality	90%	with an "unsupported_certificate" alert. Any given OID MUST NOT appear more than once in the filters list.	extension to the client. Clients MUST NOT act upon any information
371	FAIL-004	modality	90%	with an "unsupported_certificate" alert. Any given OID MUST NOT appear more than once in the filters list.	HelloRetryRequest. Each KeyShareEntry value MUST correspond to a group offered in the "supported_groups" extension and MUST appear in
372	FAIL-004	modality	90%	with an "unsupported_certificate" alert. Any given OID MUST NOT appear more than once in the filters list.	key_exchange values for each KeyShareEntry MUST be generated independently. Clients MUST NOT offer multiple KeyShareEntry values for the same group. Clients MUST NOT offer any KeyShareEntry values
373	FAIL-004	modality	90%	special anyExtendedKeyUsage OID MUST NOT be used in the request.	Servers MUST NOT send a post-handshake CertificateRequest to clients which do not offer this extension. Servers MUST NOT send this
374	FAIL-004	modality	90%	special anyExtendedKeyUsage OID MUST NOT be used in the request.	extension to the client. Clients MUST NOT act upon any information
375	FAIL-004	modality	90%	special anyExtendedKeyUsage OID MUST NOT be used in the request.	HelloRetryRequest. Each KeyShareEntry value MUST correspond to a group offered in the "supported_groups" extension and MUST appear in
376	FAIL-004	modality	90%	special anyExtendedKeyUsage OID MUST NOT be used in the request.	key_exchange values for each KeyShareEntry MUST be generated independently. Clients MUST NOT offer multiple KeyShareEntry values for the same group. Clients MUST NOT offer any KeyShareEntry values
377	FAIL-004	modality	90%	special anyExtendedKeyUsage OID MUST NOT be used in the request.	MUST verify that (1) the selected_group field corresponds to a group
378	FAIL-004	modality	90%	Servers MUST NOT send a post-handshake CertificateRequest to clients which do not offer this extension. Servers MUST NOT send this	extension to the client. Clients MUST NOT act upon any information
379	FAIL-004	modality	90%	Servers MUST NOT send a post-handshake CertificateRequest to clients which do not offer this extension. Servers MUST NOT send this	HelloRetryRequest. Each KeyShareEntry value MUST correspond to a group offered in the "supported_groups" extension and MUST appear in
380	FAIL-004	modality	90%	Servers MUST NOT send a post-handshake CertificateRequest to clients which do not offer this extension. Servers MUST NOT send this	key_exchange values for each KeyShareEntry MUST be generated independently. Clients MUST NOT offer multiple KeyShareEntry values for the same group. Clients MUST NOT offer any KeyShareEntry values
381	FAIL-004	modality	90%	Servers MUST NOT send a post-handshake CertificateRequest to clients which do not offer this extension. Servers MUST NOT send this	MUST verify that (1) the selected_group field corresponds to a group
382	FAIL-004	modality	90%	Servers MUST NOT send a post-handshake CertificateRequest to clients which do not offer this extension. Servers MUST NOT send this	the client MUST abort the handshake with an "illegal_parameter" alert. Otherwise, when sending the new ClientHello, the client MUST
383	FAIL-004	modality	90%	extension to the client. Clients MUST NOT act upon any information	HelloRetryRequest. Each KeyShareEntry value MUST correspond to a group offered in the "supported_groups" extension and MUST appear in

384	FAIL-004	modality	90%	extension to the client. Clients MUST NOT act upon any information	key_exchange values for each KeyShareEntry MUST be generated independently. Clients MUST NOT offer multiple KeyShareEntry values for the same group. Clients MUST NOT offer any KeyShareEntry values
385	FAIL-004	modality	90%	extension to the client. Clients MUST NOT act upon any information	MUST verify that (1) the selected_group field corresponds to a group
386	FAIL-004	modality	90%	extension to the client. Clients MUST NOT act upon any information	the client MUST abort the handshake with an "illegal_parameter" alert. Otherwise, when sending the new ClientHello, the client MUST
387	FAIL-004	modality	90%	extension to the client. Clients MUST NOT act upon any information	KeyShareEntry in the ServerHello. This value MUST be in the same
388	FAIL-004	modality	90%	HelloRetryRequest. Each KeyShareEntry value MUST correspond to a group offered in the "supported_groups" extension and MUST appear in	key_exchange values for each KeyShareEntry MUST be generated independently. Clients MUST NOT offer multiple KeyShareEntry values for the same group. Clients MUST NOT offer any KeyShareEntry values
389	FAIL-004	modality	90%	HelloRetryRequest. Each KeyShareEntry value MUST correspond to a group offered in the "supported_groups" extension and MUST appear in	client's "supported_groups" extension and MUST NOT send a
390	FAIL-004	modality	90%	key_exchange values for each KeyShareEntry MUST be generated independently. Clients MUST NOT offer multiple KeyShareEntry values for the same group. Clients MUST NOT offer any KeyShareEntry values	MUST verify that (1) the selected_group field corresponds to a group
391	FAIL-004	modality	90%	key_exchange values for each KeyShareEntry MUST be generated independently. Clients MUST NOT offer multiple KeyShareEntry values for the same group. Clients MUST NOT offer any KeyShareEntry values	the client MUST abort the handshake with an "illegal_parameter" alert. Otherwise, when sending the new ClientHello, the client MUST
392	FAIL-004	modality	90%	key_exchange values for each KeyShareEntry MUST be generated independently. Clients MUST NOT offer multiple KeyShareEntry values for the same group. Clients MUST NOT offer any KeyShareEntry values	KeyShareEntry in the ServerHello. This value MUST be in the same
393	FAIL-004	modality	90%	key_exchange values for each KeyShareEntry MUST be generated independently. Clients MUST NOT offer multiple KeyShareEntry values for the same group. Clients MUST NOT offer any KeyShareEntry values	client's "supported_groups" extension and MUST NOT send a
394	FAIL-004	modality	90%	key_exchange values for each KeyShareEntry MUST be generated independently. Clients MUST NOT offer multiple KeyShareEntry values for the same group. Clients MUST NOT offer any KeyShareEntry values	"key_share" extension was received by the client, the client MUST verify that the selected NamedGroup in the ServerHello is the same as that in the HelloRetryRequest. If this check fails, the client MUST abort the handshake with an "illegal_parameter" alert.
395	FAIL-004	modality	90%	key_exchange values for each KeyShareEntry MUST be generated independently. Clients MUST NOT offer multiple KeyShareEntry values for the same group. Clients MUST NOT offer any KeyShareEntry values	Peers MUST validate each other's public key Y by ensuring that $1 < Y < p-1$. This check ensures that the remote peer is properly behaved
396	FAIL-004	modality	90%	MUST verify that (1) the selected_group field corresponds to a group	client's "supported_groups" extension and MUST NOT send a
397	FAIL-004	modality	90%	the client MUST abort the handshake with an "illegal_parameter" alert. Otherwise, when sending the new ClientHello, the client MUST	client's "supported_groups" extension and MUST NOT send a
398	FAIL-004	modality	90%	KeyShareEntry in the ServerHello. This value MUST be in the same	client's "supported_groups" extension and MUST NOT send a

399	FAIL-004	modality	90%	client's "supported_groups" extension and MUST NOT send a	"key_share" extension was received by the client, the client MUST verify that the selected NamedGroup in the ServerHello is the same as that in the HelloRetryRequest. If this check fails, the client MUST abort the handshake with an "illegal_parameter" alert.
400	FAIL-004	modality	90%	client's "supported_groups" extension and MUST NOT send a	Peers MUST validate each other's public key Y by ensuring that $1 < Y < p-1$. This check ensures that the remote peer is properly behaved
401	FAIL-004	modality	90%	client's "supported_groups" extension and MUST NOT send a	For the curves secp256r1, secp384r1, and secp521r1, peers MUST validate each other's public value Q by ensuring that the point is a
402	FAIL-004	modality	90%	client's "supported_groups" extension and MUST NOT send a	In order to use PSKs, clients MUST also send a
403	FAIL-004	modality	90%	client's "supported_groups" extension and MUST NOT send a	A client MUST provide a "psk_key_exchange_modes" extension if it
404	FAIL-004	modality	90%	client's "supported_groups" extension and MUST NOT send a	servers MUST abort the handshake. Servers MUST NOT select a key
405	FAIL-004	modality	90%	"key_share" extension was received by the client, the client MUST verify that the selected NamedGroup in the ServerHello is the same as that in the HelloRetryRequest. If this check fails, the client MUST abort the handshake with an "illegal_parameter" alert.	servers MUST abort the handshake. Servers MUST NOT select a key
406	FAIL-004	modality	90%	Peers MUST validate each other's public key Y by ensuring that $1 < Y < p-1$. This check ensures that the remote peer is properly behaved	servers MUST abort the handshake. Servers MUST NOT select a key
407	FAIL-004	modality	90%	Peers MUST validate each other's public key Y by ensuring that $1 < Y < p-1$. This check ensures that the remote peer is properly behaved	The server MUST NOT send a "psk_key_exchange_modes" extension.
408	FAIL-004	modality	90%	For the curves secp256r1, secp384r1, and secp521r1, peers MUST validate each other's public value Q by ensuring that the point is a	servers MUST abort the handshake. Servers MUST NOT select a key
409	FAIL-004	modality	90%	For the curves secp256r1, secp384r1, and secp521r1, peers MUST validate each other's public value Q by ensuring that the point is a	The server MUST NOT send a "psk_key_exchange_modes" extension.
410	FAIL-004	modality	90%	In order to use PSKs, clients MUST also send a	servers MUST abort the handshake. Servers MUST NOT select a key
411	FAIL-004	modality	90%	In order to use PSKs, clients MUST also send a	The server MUST NOT send a "psk_key_exchange_modes" extension.
412	FAIL-004	modality	90%	A client MUST provide a "psk_key_exchange_modes" extension if it	servers MUST abort the handshake. Servers MUST NOT select a key
413	FAIL-004	modality	90%	A client MUST provide a "psk_key_exchange_modes" extension if it	The server MUST NOT send a "psk_key_exchange_modes" extension.
414	FAIL-004	modality	90%	servers MUST abort the handshake. Servers MUST NOT select a key	The server MUST NOT send a "psk_key_exchange_modes" extension.
415	FAIL-004	modality	90%	servers MUST abort the handshake. Servers MUST NOT select a key	Any future values that are allocated must ensure that the transmitted
416	FAIL-004	modality	90%	servers MUST abort the handshake. Servers MUST NOT select a key	client opts to do so, it MUST supply both the "pre_shared_key" and
417	FAIL-004	modality	90%	servers MUST abort the handshake. Servers MUST NOT select a key	early data MUST be the first PSK listed in the client's
418	FAIL-004	modality	90%	servers MUST abort the handshake. Servers MUST NOT select a key	For PSKs provisioned via NewSessionTicket, a server MUST validate
419	FAIL-004	modality	90%	servers MUST abort the handshake. Servers MUST NOT select a key	A server which receives an "early_data" extension MUST behave in one
420	FAIL-004	modality	90%	servers MUST abort the handshake. Servers MUST NOT select a key	with a HelloRetryRequest. A client MUST NOT include the

421	FAIL-004	modality	90%	The server MUST NOT send a "psk_key_exchange_modes" extension.	Any future values that are allocated must ensure that the transmitted
422	FAIL-004	modality	90%	The server MUST NOT send a "psk_key_exchange_modes" extension.	client opts to do so, it MUST supply both the "pre_shared_key" and
423	FAIL-004	modality	90%	The server MUST NOT send a "psk_key_exchange_modes" extension.	early data MUST be the first PSK listed in the client's
424	FAIL-004	modality	90%	The server MUST NOT send a "psk_key_exchange_modes" extension.	For PSKs provisioned via NewSessionTicket, a server MUST validate
425	FAIL-004	modality	90%	The server MUST NOT send a "psk_key_exchange_modes" extension.	A server which receives an "early_data" extension MUST behave in one
426	FAIL-004	modality	90%	The server MUST NOT send a "psk_key_exchange_modes" extension.	with a HelloRetryRequest. A client MUST NOT include the
427	FAIL-004	modality	90%	The server MUST NOT send a "psk_key_exchange_modes" extension.	In order to accept early data, the server MUST have accepted a PSK
428	FAIL-004	modality	90%	The server MUST NOT send a "psk_key_exchange_modes" extension.	"pre_shared_key" extension. In addition, it MUST verify that the
429	FAIL-004	modality	90%	Any future values that are allocated must ensure that the transmitted	with a HelloRetryRequest. A client MUST NOT include the
430	FAIL-004	modality	90%	Any future values that are allocated must ensure that the transmitted	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the
431	FAIL-004	modality	90%	client opts to do so, it MUST supply both the "pre_shared_key" and	with a HelloRetryRequest. A client MUST NOT include the
432	FAIL-004	modality	90%	client opts to do so, it MUST supply both the "pre_shared_key" and	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the
433	FAIL-004	modality	90%	early data MUST be the first PSK listed in the client's	with a HelloRetryRequest. A client MUST NOT include the
434	FAIL-004	modality	90%	early data MUST be the first PSK listed in the client's	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the
435	FAIL-004	modality	90%	For PSKs provisioned via NewSessionTicket, a server MUST validate	with a HelloRetryRequest. A client MUST NOT include the
436	FAIL-004	modality	90%	For PSKs provisioned via NewSessionTicket, a server MUST validate	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the
437	FAIL-004	modality	90%	A server which receives an "early_data" extension MUST behave in one	with a HelloRetryRequest. A client MUST NOT include the
438	FAIL-004	modality	90%	A server which receives an "early_data" extension MUST behave in one	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the
439	FAIL-004	modality	90%	A server which receives an "early_data" extension MUST behave in one	is appropriate. A TLS implementation MUST NOT automatically resend
440	FAIL-004	modality	90%	with a HelloRetryRequest. A client MUST NOT include the	In order to accept early data, the server MUST have accepted a PSK
441	FAIL-004	modality	90%	with a HelloRetryRequest. A client MUST NOT include the	"pre_shared_key" extension. In addition, it MUST verify that the
442	FAIL-004	modality	90%	with a HelloRetryRequest. A client MUST NOT include the	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the
443	FAIL-004	modality	90%	with a HelloRetryRequest. A client MUST NOT include the	rejects it, the server will generally not have the 0-RTT record protection keys and must instead use trial decryption (either with
444	FAIL-004	modality	90%	with a HelloRetryRequest. A client MUST NOT include the	MUST comply with the same error-handling requirements specified for
445	FAIL-004	modality	90%	with a HelloRetryRequest. A client MUST NOT include the	"early_data" extension, it MUST terminate the connection with a

446	FAIL-004	modality	90%	with a HelloRetryRequest. A client MUST NOT include the	is appropriate. A TLS implementation MUST NOT automatically resend
447	FAIL-004	modality	90%	with a HelloRetryRequest. A client MUST NOT include the	For externally established PSKs, the Hash algorithm MUST be set when
448	FAIL-004	modality	90%	In order to accept early data, the server MUST have accepted a PSK	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the
449	FAIL-004	modality	90%	In order to accept early data, the server MUST have accepted a PSK	is appropriate. A TLS implementation MUST NOT automatically resend
450	FAIL-004	modality	90%	"pre_shared_key" extension. In addition, it MUST verify that the	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the
451	FAIL-004	modality	90%	"pre_shared_key" extension. In addition, it MUST verify that the	is appropriate. A TLS implementation MUST NOT automatically resend
452	FAIL-004	modality	90%	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the	rejects it, the server will generally not have the 0-RTT record protection keys and must instead use trial decryption (either with
453	FAIL-004	modality	90%	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the	MUST comply with the same error-handling requirements specified for
454	FAIL-004	modality	90%	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the	"early_data" extension, it MUST terminate the connection with a
455	FAIL-004	modality	90%	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the	is appropriate. A TLS implementation MUST NOT automatically resend
456	FAIL-004	modality	90%	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the	For externally established PSKs, the Hash algorithm MUST be set when
457	FAIL-004	modality	90%	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the	Prior to accepting PSK key establishment, the server MUST validate
458	FAIL-004	modality	90%	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the	value is not present or does not validate, the server MUST abort the
459	FAIL-004	modality	90%	If any of these checks fail, the server MUST NOT respond with the extension and must discard all the first-flight data using one of the	Clients MUST verify that the server's selected_identity is within the
460	FAIL-004	modality	90%	rejects it, the server will generally not have the 0-RTT record protection keys and must instead use trial decryption (either with	is appropriate. A TLS implementation MUST NOT automatically resend
461	FAIL-004	modality	90%	MUST comply with the same error-handling requirements specified for	is appropriate. A TLS implementation MUST NOT automatically resend
462	FAIL-004	modality	90%	"early_data" extension, it MUST terminate the connection with a	is appropriate. A TLS implementation MUST NOT automatically resend
463	FAIL-004	modality	90%	is appropriate. A TLS implementation MUST NOT automatically resend	For externally established PSKs, the Hash algorithm MUST be set when
464	FAIL-004	modality	90%	is appropriate. A TLS implementation MUST NOT automatically resend	Prior to accepting PSK key establishment, the server MUST validate
465	FAIL-004	modality	90%	is appropriate. A TLS implementation MUST NOT automatically resend	value is not present or does not validate, the server MUST abort the
466	FAIL-004	modality	90%	is appropriate. A TLS implementation MUST NOT automatically resend	Clients MUST verify that the server's selected_identity is within the
467	FAIL-004	modality	90%	is appropriate. A TLS implementation MUST NOT automatically resend	consistent, the client MUST abort the handshake with an

468	FAIL-004	modality	90%	is appropriate. A TLS implementation MUST NOT automatically resend	If the server supplies an "early_data" extension, the client MUST verify that the server's selected_identity is 0. If any other value is returned, the client MUST abort the handshake with an
469	FAIL-004	modality	90%	is appropriate. A TLS implementation MUST NOT automatically resend	The "pre_shared_key" extension MUST be the last extension in the
470	FAIL-004	modality	90%	Prior to accepting PSK key establishment, the server MUST validate	receipt of the NewSessionTicket message. Clients MUST NOT attempt to use tickets which have ages greater than the "ticket_lifetime" value
471	FAIL-004	modality	90%	value is not present or does not validate, the server MUST abort the	receipt of the NewSessionTicket message. Clients MUST NOT attempt to use tickets which have ages greater than the "ticket_lifetime" value
472	FAIL-004	modality	90%	Clients MUST verify that the server's selected_identity is within the	receipt of the NewSessionTicket message. Clients MUST NOT attempt to use tickets which have ages greater than the "ticket_lifetime" value
473	FAIL-004	modality	90%	consistent, the client MUST abort the handshake with an	receipt of the NewSessionTicket message. Clients MUST NOT attempt to use tickets which have ages greater than the "ticket_lifetime" value
474	FAIL-004	modality	90%	If the server supplies an "early_data" extension, the client MUST verify that the server's selected_identity is 0. If any other value is returned, the client MUST abort the handshake with an	receipt of the NewSessionTicket message. Clients MUST NOT attempt to use tickets which have ages greater than the "ticket_lifetime" value
475	FAIL-004	modality	90%	The "pre_shared_key" extension MUST be the last extension in the	receipt of the NewSessionTicket message. Clients MUST NOT attempt to use tickets which have ages greater than the "ticket_lifetime" value
476	FAIL-004	modality	90%	Servers MUST check that it is the last extension and otherwise fail	receipt of the NewSessionTicket message. Clients MUST NOT attempt to use tickets which have ages greater than the "ticket_lifetime" value
477	FAIL-004	modality	90%	receipt of the NewSessionTicket message. Clients MUST NOT attempt to use tickets which have ages greater than the "ticket_lifetime" value	when accepting "early_data", servers MUST process the client's
478	FAIL-004	modality	90%	receipt of the NewSessionTicket message. Clients MUST NOT attempt to use tickets which have ages greater than the "ticket_lifetime" value	In all handshakes, the server MUST send the EncryptedExtensions
479	FAIL-004	modality	90%	receipt of the NewSessionTicket message. Clients MUST NOT attempt to use tickets which have ages greater than the "ticket_lifetime" value	certificates. The client MUST check EncryptedExtensions for the presence of any forbidden extensions and if any are found MUST abort
480	FAIL-004	modality	90%	receipt of the NewSessionTicket message. Clients MUST NOT attempt to use tickets which have ages greater than the "ticket_lifetime" value	request a certificate from the client. This message, if sent, MUST follow EncryptedExtensions.
481	FAIL-004	modality	90%	receipt of the NewSessionTicket message. Clients MUST NOT attempt to use tickets which have ages greater than the "ticket_lifetime" value	MUST be specified, and other extensions may optionally be included
482	FAIL-004	modality	90%	when accepting "early_data", servers MUST process the client's	Servers which are authenticating with a PSK MUST NOT send the
483	FAIL-004	modality	90%	In all handshakes, the server MUST send the EncryptedExtensions	Servers which are authenticating with a PSK MUST NOT send the
484	FAIL-004	modality	90%	certificates. The client MUST check EncryptedExtensions for the presence of any forbidden extensions and if any are found MUST abort	Servers which are authenticating with a PSK MUST NOT send the
485	FAIL-004	modality	90%	request a certificate from the client. This message, if sent, MUST follow EncryptedExtensions.	Servers which are authenticating with a PSK MUST NOT send the
486	FAIL-004	modality	90%	MUST be specified, and other extensions may optionally be included	Servers which are authenticating with a PSK MUST NOT send the
487	FAIL-004	modality	90%	Servers which are authenticating with a PSK MUST NOT send the	The server MUST send a Certificate message whenever the agreed-upon

488	FAIL-004	modality	90%	Servers which are authenticating with a PSK MUST NOT send the	The client MUST send a Certificate message if and only if the server
489	FAIL-004	modality	90%	Servers which are authenticating with a PSK MUST NOT send the	suitable certificate is available, the client MUST send a Certificate
490	FAIL-004	modality	90%	Servers which are authenticating with a PSK MUST NOT send the	field having length 0). A Finished message MUST be sent regardless
491	FAIL-004	modality	90%	Servers which are authenticating with a PSK MUST NOT send the	well. Extensions in the Certificate message from the server MUST correspond to ones from the ClientHello message. Extensions in the Certificate message from the client MUST correspond to
492	FAIL-004	modality	90%	Servers which are authenticating with a PSK MUST NOT send the	certificate. The sender's certificate MUST come in the first
493	FAIL-004	modality	90%	Servers which are authenticating with a PSK MUST NOT send the	certificate_list MUST contain no more than one CertificateEntry,
494	FAIL-004	modality	90%	The server MUST send a Certificate message whenever the agreed-upon	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.
495	FAIL-004	modality	90%	The client MUST send a Certificate message if and only if the server	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.
496	FAIL-004	modality	90%	suitable certificate is available, the client MUST send a Certificate	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.
497	FAIL-004	modality	90%	suitable certificate is available, the client MUST send a Certificate	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,
498	FAIL-004	modality	90%	field having length 0). A Finished message MUST be sent regardless	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.
499	FAIL-004	modality	90%	field having length 0). A Finished message MUST be sent regardless	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,
500	FAIL-004	modality	90%	well. Extensions in the Certificate message from the server MUST correspond to ones from the ClientHello message. Extensions in the Certificate message from the client MUST correspond to	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.
501	FAIL-004	modality	90%	well. Extensions in the Certificate message from the server MUST correspond to ones from the ClientHello message. Extensions in the Certificate message from the client MUST correspond to	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,
502	FAIL-004	modality	90%	certificate. The sender's certificate MUST come in the first	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.
503	FAIL-004	modality	90%	certificate. The sender's certificate MUST come in the first	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,
504	FAIL-004	modality	90%	certificate_list MUST contain no more than one CertificateEntry,	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.
505	FAIL-004	modality	90%	certificate_list MUST contain no more than one CertificateEntry,	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,
506	FAIL-004	modality	90%	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.	The server's certificate_list MUST always be non-empty. A client will send an empty certificate_list if it does not have an
507	FAIL-004	modality	90%	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.	"status_request" extension from the server MUST be a

508	FAIL-004	modality	90%	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,
509	FAIL-004	modality	90%	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.	CertificateRequest, or Certificate messages. TLS 1.3 servers MUST be
510	FAIL-004	modality	90%	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.	response, the body of its "status_request" extension MUST be a
511	FAIL-004	modality	90%	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.	- The certificate type MUST be X.509v3 [RFC5280], unless explicitly
512	FAIL-004	modality	90%	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.	restrictions) MUST be compatible with the selected authentication
513	FAIL-004	modality	90%	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.	- The certificate MUST allow the key to be used for signing (i.e., the digitalSignature bit MUST be set if the Key Usage extension is
514	FAIL-004	modality	90%	The OpenPGP certificate type [RFC6091] MUST NOT be used with TLS 1.3.	All certificates provided by the server MUST be signed by a signature
515	FAIL-004	modality	90%	The server's certificate_list MUST always be non-empty. A client will send an empty certificate_list if it does not have an	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,
516	FAIL-004	modality	90%	The server's certificate_list MUST always be non-empty. A client will send an empty certificate_list if it does not have an	permits it, and MUST NOT do so otherwise.
517	FAIL-004	modality	90%	"status_request" extension from the server MUST be a	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,
518	FAIL-004	modality	90%	"status_request" extension from the server MUST be a	permits it, and MUST NOT do so otherwise.
519	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,	CertificateRequest, or Certificate messages. TLS 1.3 servers MUST be
520	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,	response, the body of its "status_request" extension MUST be a
521	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,	- The certificate type MUST be X.509v3 [RFC5280], unless explicitly
522	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,	restrictions) MUST be compatible with the selected authentication
523	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,	- The certificate MUST allow the key to be used for signing (i.e., the digitalSignature bit MUST be set if the Key Usage extension is
524	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,	All certificates provided by the server MUST be signed by a signature

525	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,	permits it, and MUST NOT do so otherwise.
526	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,	certificates and decides to abort the handshake, then it MUST abort
527	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,	- The certificate type MUST be X.509v3 [RFC5280], unless explicitly
528	FAIL-004	modality	90%	TLS 1.3 servers MUST NOT act upon its presence or information in it when processing ClientHello messages; in particular, they MUST NOT send the status_request_v2 extension in the EncryptedExtensions,	- The certificates MUST be signed using an acceptable signature
529	FAIL-004	modality	90%	CertificateRequest, or Certificate messages. TLS 1.3 servers MUST be	permits it, and MUST NOT do so otherwise.
530	FAIL-004	modality	90%	response, the body of its "status_request" extension MUST be a	permits it, and MUST NOT do so otherwise.
531	FAIL-004	modality	90%	- The certificate type MUST be X.509v3 [RFC5280], unless explicitly	permits it, and MUST NOT do so otherwise.
532	FAIL-004	modality	90%	restrictions) MUST be compatible with the selected authentication	permits it, and MUST NOT do so otherwise.
533	FAIL-004	modality	90%	- The certificate MUST allow the key to be used for signing (i.e., the digitalSignature bit MUST be set if the Key Usage extension is	permits it, and MUST NOT do so otherwise.
534	FAIL-004	modality	90%	All certificates provided by the server MUST be signed by a signature	permits it, and MUST NOT do so otherwise.
535	FAIL-004	modality	90%	permits it, and MUST NOT do so otherwise.	certificates and decides to abort the handshake, then it MUST abort
536	FAIL-004	modality	90%	permits it, and MUST NOT do so otherwise.	- The certificate type MUST be X.509v3 [RFC5280], unless explicitly
537	FAIL-004	modality	90%	permits it, and MUST NOT do so otherwise.	- The certificates MUST be signed using an acceptable signature
538	FAIL-004	modality	90%	permits it, and MUST NOT do so otherwise.	"oid_filters" extension, the end-entity certificate MUST match the
539	FAIL-004	modality	90%	permits it, and MUST NOT do so otherwise.	If the server supplies an empty Certificate message, the client MUST abort the handshake with a "decode_error" alert.
540	FAIL-004	modality	90%	permits it, and MUST NOT do so otherwise.	validate using any signature algorithm using an MD5 hash MUST abort
541	FAIL-004	modality	90%	permits it, and MUST NOT do so otherwise.	up to this point. Servers MUST send this message when authenticating via a certificate. Clients MUST send this message whenever
542	FAIL-004	modality	90%	permits it, and MUST NOT do so otherwise.	is non-empty). When sent, this message MUST appear immediately after
543	FAIL-004	modality	90%	permits it, and MUST NOT do so otherwise.	algorithm MUST be one offered in the client's "signature_algorithms"
544	FAIL-004	modality	90%	permits it, and MUST NOT do so otherwise.	MUST be one of those present in the supported_signature_algorithms
545	FAIL-004	modality	90%	- The certificate type MUST be X.509v3 [RFC5280], unless explicitly	MUST NOT be used in any signatures of CertificateVerify messages.
546	FAIL-004	modality	90%	- The certificates MUST be signed using an acceptable signature	MUST NOT be used in any signatures of CertificateVerify messages.

547	FAIL-004	modality	90%	"oid_filters" extension, the end-entity certificate MUST match the	MUST NOT be used in any signatures of CertificateVerify messages.
548	FAIL-004	modality	90%	If the server supplies an empty Certificate message, the client MUST abort the handshake with a "decode_error" alert.	MUST NOT be used in any signatures of CertificateVerify messages.
549	FAIL-004	modality	90%	validate using any signature algorithm using an MD5 hash MUST abort	MUST NOT be used in any signatures of CertificateVerify messages.
550	FAIL-004	modality	90%	up to this point. Servers MUST send this message when authenticating via a certificate. Clients MUST send this message whenever	MUST NOT be used in any signatures of CertificateVerify messages.
551	FAIL-004	modality	90%	is non-empty). When sent, this message MUST appear immediately after	MUST NOT be used in any signatures of CertificateVerify messages.
552	FAIL-004	modality	90%	algorithm MUST be one offered in the client's "signature_algorithms"	MUST NOT be used in any signatures of CertificateVerify messages.
553	FAIL-004	modality	90%	MUST be one of those present in the supported_signature_algorithms	MUST NOT be used in any signatures of CertificateVerify messages.
554	FAIL-004	modality	90%	MUST be one of those present in the supported_signature_algorithms	extension in EncryptedExtensions, then the client MUST NOT send an
555	FAIL-004	modality	90%	In addition, the signature algorithm MUST be compatible with the key in the sender's end-entity certificate. RSA signatures MUST use an	MUST NOT be used in any signatures of CertificateVerify messages.
556	FAIL-004	modality	90%	In addition, the signature algorithm MUST be compatible with the key in the sender's end-entity certificate. RSA signatures MUST use an	extension in EncryptedExtensions, then the client MUST NOT send an
557	FAIL-004	modality	90%	In addition, the signature algorithm MUST be compatible with the key in the sender's end-entity certificate. RSA signatures MUST use an	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This
558	FAIL-004	modality	90%	MUST NOT be used in any signatures of CertificateVerify messages.	The receiver of a CertificateVerify message MUST verify the signature
559	FAIL-004	modality	90%	MUST NOT be used in any signatures of CertificateVerify messages.	If the verification fails, the receiver MUST terminate the handshake
560	FAIL-004	modality	90%	MUST NOT be used in any signatures of CertificateVerify messages.	Recipients of Finished messages MUST verify that the contents are correct and if incorrect MUST terminate the connection with a
561	FAIL-004	modality	90%	MUST NOT be used in any signatures of CertificateVerify messages.	Any records following a Finished message MUST be encrypted under the
562	FAIL-004	modality	90%	MUST NOT be used in any signatures of CertificateVerify messages.	the client MUST send an EndOfEarlyData message after receiving the
563	FAIL-004	modality	90%	MUST NOT be used in any signatures of CertificateVerify messages.	extension in EncryptedExtensions, then the client MUST NOT send an
564	FAIL-004	modality	90%	MUST NOT be used in any signatures of CertificateVerify messages.	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This
565	FAIL-004	modality	90%	MUST NOT be used in any signatures of CertificateVerify messages.	Any ticket MUST only be resumed with a cipher suite that has the same
566	FAIL-004	modality	90%	The receiver of a CertificateVerify message MUST verify the signature	extension in EncryptedExtensions, then the client MUST NOT send an
567	FAIL-004	modality	90%	The receiver of a CertificateVerify message MUST verify the signature	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This
568	FAIL-004	modality	90%	If the verification fails, the receiver MUST terminate the handshake	extension in EncryptedExtensions, then the client MUST NOT send an
569	FAIL-004	modality	90%	If the verification fails, the receiver MUST terminate the handshake	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This
570	FAIL-004	modality	90%	Recipients of Finished messages MUST verify that the contents are correct and if incorrect MUST terminate the connection with a	extension in EncryptedExtensions, then the client MUST NOT send an

571	FAIL-004	modality	90%	Recipients of Finished messages MUST verify that the contents are correct and if incorrect MUST terminate the connection with a	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This
572	FAIL-004	modality	90%	Recipients of Finished messages MUST verify that the contents are correct and if incorrect MUST terminate the connection with a	issuance. Servers MUST NOT use any value greater than
573	FAIL-004	modality	90%	Any records following a Finished message MUST be encrypted under the	extension in EncryptedExtensions, then the client MUST NOT send an
574	FAIL-004	modality	90%	Any records following a Finished message MUST be encrypted under the	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This
575	FAIL-004	modality	90%	Any records following a Finished message MUST be encrypted under the	issuance. Servers MUST NOT use any value greater than
576	FAIL-004	modality	90%	Any records following a Finished message MUST be encrypted under the	ticket should be discarded immediately. Clients MUST NOT cache
577	FAIL-004	modality	90%	the client MUST send an EndOfEarlyData message after receiving the	extension in EncryptedExtensions, then the client MUST NOT send an
578	FAIL-004	modality	90%	the client MUST send an EndOfEarlyData message after receiving the	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This
579	FAIL-004	modality	90%	the client MUST send an EndOfEarlyData message after receiving the	issuance. Servers MUST NOT use any value greater than
580	FAIL-004	modality	90%	the client MUST send an EndOfEarlyData message after receiving the	ticket should be discarded immediately. Clients MUST NOT cache
581	FAIL-004	modality	90%	extension in EncryptedExtensions, then the client MUST NOT send an	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This
582	FAIL-004	modality	90%	extension in EncryptedExtensions, then the client MUST NOT send an	Any ticket MUST only be resumed with a cipher suite that has the same
583	FAIL-004	modality	90%	extension in EncryptedExtensions, then the client MUST NOT send an	Clients MUST only resume if the new SNI value is valid for the server
584	FAIL-004	modality	90%	extension in EncryptedExtensions, then the client MUST NOT send an	implementations MUST use the value sent in the resumption ClientHello
585	FAIL-004	modality	90%	extension in EncryptedExtensions, then the client MUST NOT send an	issuance. Servers MUST NOT use any value greater than
586	FAIL-004	modality	90%	extension in EncryptedExtensions, then the client MUST NOT send an	ticket should be discarded immediately. Clients MUST NOT cache
587	FAIL-004	modality	90%	extension in EncryptedExtensions, then the client MUST NOT send an	transmitted by the client. The server MUST generate a fresh value
588	FAIL-004	modality	90%	extension in EncryptedExtensions, then the client MUST NOT send an	CertificateRequest message. The client MUST respond with the
589	FAIL-004	modality	90%	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This	Any ticket MUST only be resumed with a cipher suite that has the same
590	FAIL-004	modality	90%	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This	Clients MUST only resume if the new SNI value is valid for the server
591	FAIL-004	modality	90%	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This	implementations MUST use the value sent in the resumption ClientHello
592	FAIL-004	modality	90%	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This	issuance. Servers MUST NOT use any value greater than
593	FAIL-004	modality	90%	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This	ticket should be discarded immediately. Clients MUST NOT cache

594	FAIL-004	modality	90%	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This	transmitted by the client. The server MUST generate a fresh value
595	FAIL-004	modality	90%	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This	CertificateRequest message. The client MUST respond with the
596	FAIL-004	modality	90%	Servers MUST NOT send this message, and clients receiving it MUST terminate the connection with an "unexpected_message" alert. This	chooses to authenticate, it MUST send Certificate, CertificateVerify,
597	FAIL-004	modality	90%	Any ticket MUST only be resumed with a cipher suite that has the same	issuance. Servers MUST NOT use any value greater than
598	FAIL-004	modality	90%	Any ticket MUST only be resumed with a cipher suite that has the same	ticket should be discarded immediately. Clients MUST NOT cache
599	FAIL-004	modality	90%	Clients MUST only resume if the new SNI value is valid for the server	issuance. Servers MUST NOT use any value greater than
600	FAIL-004	modality	90%	Clients MUST only resume if the new SNI value is valid for the server	ticket should be discarded immediately. Clients MUST NOT cache
601	FAIL-004	modality	90%	implementations MUST use the value sent in the resumption ClientHello	issuance. Servers MUST NOT use any value greater than
602	FAIL-004	modality	90%	implementations MUST use the value sent in the resumption ClientHello	ticket should be discarded immediately. Clients MUST NOT cache
603	FAIL-004	modality	90%	issuance. Servers MUST NOT use any value greater than	ticket should be discarded immediately. Clients MUST NOT cache
604	FAIL-004	modality	90%	issuance. Servers MUST NOT use any value greater than	transmitted by the client. The server MUST generate a fresh value
605	FAIL-004	modality	90%	issuance. Servers MUST NOT use any value greater than	CertificateRequest message. The client MUST respond with the
606	FAIL-004	modality	90%	issuance. Servers MUST NOT use any value greater than	chooses to authenticate, it MUST send Certificate, CertificateVerify,
607	FAIL-004	modality	90%	issuance. Servers MUST NOT use any value greater than	and Finished. If it declines, it MUST send a Certificate message
608	FAIL-004	modality	90%	issuance. Servers MUST NOT use any value greater than	messages for a given response MUST appear consecutively on the wire
609	FAIL-004	modality	90%	issuance. Servers MUST NOT use any value greater than	servers MUST be prepared for some delay, including receiving an
610	FAIL-004	modality	90%	ticket should be discarded immediately. Clients MUST NOT cache	transmitted by the client. The server MUST generate a fresh value
611	FAIL-004	modality	90%	ticket should be discarded immediately. Clients MUST NOT cache	CertificateRequest message. The client MUST respond with the
612	FAIL-004	modality	90%	ticket should be discarded immediately. Clients MUST NOT cache	chooses to authenticate, it MUST send Certificate, CertificateVerify,
613	FAIL-004	modality	90%	ticket should be discarded immediately. Clients MUST NOT cache	and Finished. If it declines, it MUST send a Certificate message
614	FAIL-004	modality	90%	ticket should be discarded immediately. Clients MUST NOT cache	messages for a given response MUST appear consecutively on the wire
615	FAIL-004	modality	90%	ticket should be discarded immediately. Clients MUST NOT cache	servers MUST be prepared for some delay, including receiving an
616	FAIL-004	modality	90%	ticket should be discarded immediately. Clients MUST NOT cache	message MUST terminate the connection with an "unexpected_message" alert. After sending a KeyUpdate message, the sender SHALL send all
617	FAIL-004	modality	90%	Both sender and receiver MUST encrypt their KeyUpdate messages with the old keys. Additionally, both sides MUST enforce that a KeyUpdate	- Handshake messages MUST NOT be interleaved with other record
618	FAIL-004	modality	90%	Both sender and receiver MUST encrypt their KeyUpdate messages with the old keys. Additionally, both sides MUST enforce that a KeyUpdate	records, there MUST NOT be any other records between them.

619	FAIL-004	modality	90%	and before the peer's Finished message has been received and MUST	- Handshake messages MUST NOT be interleaved with other record
620	FAIL-004	modality	90%	and before the peer's Finished message has been received and MUST	records, there MUST NOT be any other records between them.
621	FAIL-004	modality	90%	and before the peer's Finished message has been received and MUST	- Handshake messages MUST NOT span key changes. Implementations
622	FAIL-004	modality	90%	which receives a protected change_cipher_spec record MUST abort the handshake with an "unexpected_message" alert. If an implementation	- Handshake messages MUST NOT be interleaved with other record
623	FAIL-004	modality	90%	which receives a protected change_cipher_spec record MUST abort the handshake with an "unexpected_message" alert. If an implementation	records, there MUST NOT be any other records between them.
624	FAIL-004	modality	90%	which receives a protected change_cipher_spec record MUST abort the handshake with an "unexpected_message" alert. If an implementation	- Handshake messages MUST NOT span key changes. Implementations
625	FAIL-004	modality	90%	which receives a protected change_cipher_spec record MUST abort the handshake with an "unexpected_message" alert. If an implementation	Implementations MUST NOT send zero-length fragments of Handshake
626	FAIL-004	modality	90%	ClientHello message or after the peer's Finished message, it MUST be	- Handshake messages MUST NOT be interleaved with other record
627	FAIL-004	modality	90%	ClientHello message or after the peer's Finished message, it MUST be	records, there MUST NOT be any other records between them.
628	FAIL-004	modality	90%	ClientHello message or after the peer's Finished message, it MUST be	- Handshake messages MUST NOT span key changes. Implementations
629	FAIL-004	modality	90%	ClientHello message or after the peer's Finished message, it MUST be	Implementations MUST NOT send zero-length fragments of Handshake
630	FAIL-004	modality	90%	ClientHello message or after the peer's Finished message, it MUST be	multiple alert messages MUST NOT be coalesced into a single
631	FAIL-004	modality	90%	implementation receives an unexpected record type, it MUST terminate	- Handshake messages MUST NOT be interleaved with other record
632	FAIL-004	modality	90%	implementation receives an unexpected record type, it MUST terminate	records, there MUST NOT be any other records between them.
633	FAIL-004	modality	90%	implementation receives an unexpected record type, it MUST terminate	- Handshake messages MUST NOT span key changes. Implementations
634	FAIL-004	modality	90%	implementation receives an unexpected record type, it MUST terminate	Implementations MUST NOT send zero-length fragments of Handshake
635	FAIL-004	modality	90%	implementation receives an unexpected record type, it MUST terminate	multiple alert messages MUST NOT be coalesced into a single
636	FAIL-004	modality	90%	ContentType. Any future content types MUST specify appropriate	- Handshake messages MUST NOT be interleaved with other record
637	FAIL-004	modality	90%	ContentType. Any future content types MUST specify appropriate	records, there MUST NOT be any other records between them.
638	FAIL-004	modality	90%	ContentType. Any future content types MUST specify appropriate	- Handshake messages MUST NOT span key changes. Implementations
639	FAIL-004	modality	90%	ContentType. Any future content types MUST specify appropriate	Implementations MUST NOT send zero-length fragments of Handshake
640	FAIL-004	modality	90%	ContentType. Any future content types MUST specify appropriate	multiple alert messages MUST NOT be coalesced into a single
641	FAIL-004	modality	90%	ContentType. Any future content types MUST specify appropriate	TLSPplaintext.fragment. The length MUST NOT exceed 2 ¹⁴ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.
642	FAIL-004	modality	90%	- Handshake messages MUST NOT be interleaved with other record	records, there MUST NOT be any other records between them.
643	FAIL-004	modality	90%	- Handshake messages MUST NOT be interleaved with other record	- Handshake messages MUST NOT span key changes. Implementations

644	FAIL-004	modality	90%	- Handshake messages MUST NOT be interleaved with other record	MUST verify that all messages immediately preceding a key change align with a record boundary; if not, then they MUST terminate the
645	FAIL-004	modality	90%	- Handshake messages MUST NOT be interleaved with other record	MUST send these messages in alignment with a record boundary.
646	FAIL-004	modality	90%	- Handshake messages MUST NOT be interleaved with other record	Implementations MUST NOT send zero-length fragments of Handshake
647	FAIL-004	modality	90%	- Handshake messages MUST NOT be interleaved with other record	multiple alert messages MUST NOT be coalesced into a single
648	FAIL-004	modality	90%	- Handshake messages MUST NOT be interleaved with other record	legacy_record_version: MUST be set to 0x0303 for all records
649	FAIL-004	modality	90%	- Handshake messages MUST NOT be interleaved with other record	field is deprecated and MUST be ignored for all purposes.
650	FAIL-004	modality	90%	- Handshake messages MUST NOT be interleaved with other record	TLSPplaintext.fragment. The length MUST NOT exceed 2 ¹⁴ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.
651	FAIL-004	modality	90%	- Handshake messages MUST NOT be interleaved with other record	ClientHello or a ServerHello MUST have version 0x0303 (reflecting)
652	FAIL-004	modality	90%	records, there MUST NOT be any other records between them.	- Handshake messages MUST NOT span key changes. Implementations
653	FAIL-004	modality	90%	records, there MUST NOT be any other records between them.	MUST verify that all messages immediately preceding a key change align with a record boundary; if not, then they MUST terminate the
654	FAIL-004	modality	90%	records, there MUST NOT be any other records between them.	MUST send these messages in alignment with a record boundary.
655	FAIL-004	modality	90%	records, there MUST NOT be any other records between them.	Implementations MUST NOT send zero-length fragments of Handshake
656	FAIL-004	modality	90%	records, there MUST NOT be any other records between them.	multiple alert messages MUST NOT be coalesced into a single
657	FAIL-004	modality	90%	records, there MUST NOT be any other records between them.	legacy_record_version: MUST be set to 0x0303 for all records
658	FAIL-004	modality	90%	records, there MUST NOT be any other records between them.	field is deprecated and MUST be ignored for all purposes.
659	FAIL-004	modality	90%	records, there MUST NOT be any other records between them.	TLSPplaintext.fragment. The length MUST NOT exceed 2 ¹⁴ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.
660	FAIL-004	modality	90%	records, there MUST NOT be any other records between them.	ClientHello or a ServerHello MUST have version 0x0303 (reflecting)
661	FAIL-004	modality	90%	- Handshake messages MUST NOT span key changes. Implementations	MUST verify that all messages immediately preceding a key change align with a record boundary; if not, then they MUST terminate the
662	FAIL-004	modality	90%	- Handshake messages MUST NOT span key changes. Implementations	MUST send these messages in alignment with a record boundary.
663	FAIL-004	modality	90%	- Handshake messages MUST NOT span key changes. Implementations	Implementations MUST NOT send zero-length fragments of Handshake
664	FAIL-004	modality	90%	- Handshake messages MUST NOT span key changes. Implementations	multiple alert messages MUST NOT be coalesced into a single
665	FAIL-004	modality	90%	- Handshake messages MUST NOT span key changes. Implementations	legacy_record_version: MUST be set to 0x0303 for all records
666	FAIL-004	modality	90%	- Handshake messages MUST NOT span key changes. Implementations	field is deprecated and MUST be ignored for all purposes.
667	FAIL-004	modality	90%	- Handshake messages MUST NOT span key changes. Implementations	TLSPplaintext.fragment. The length MUST NOT exceed 2 ¹⁴ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.

668	FAIL-004	modality	90%	- Handshake messages MUST NOT span key changes. Implementations	ClientHello or a ServerHello MUST have version 0x0303 (reflecting)
669	FAIL-004	modality	90%	- Handshake messages MUST NOT span key changes. Implementations	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with
670	FAIL-004	modality	90%	MUST verify that all messages immediately preceding a key change align with a record boundary; if not, then they MUST terminate the	Implementations MUST NOT send zero-length fragments of Handshake
671	FAIL-004	modality	90%	MUST verify that all messages immediately preceding a key change align with a record boundary; if not, then they MUST terminate the	multiple alert messages MUST NOT be coalesced into a single
672	FAIL-004	modality	90%	MUST verify that all messages immediately preceding a key change align with a record boundary; if not, then they MUST terminate the	TLSPplaintext.fragment. The length MUST NOT exceed 2^{14} bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.
673	FAIL-004	modality	90%	MUST verify that all messages immediately preceding a key change align with a record boundary; if not, then they MUST terminate the	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with
674	FAIL-004	modality	90%	MUST send these messages in alignment with a record boundary.	Implementations MUST NOT send zero-length fragments of Handshake
675	FAIL-004	modality	90%	MUST send these messages in alignment with a record boundary.	multiple alert messages MUST NOT be coalesced into a single
676	FAIL-004	modality	90%	MUST send these messages in alignment with a record boundary.	TLSPplaintext.fragment. The length MUST NOT exceed 2^{14} bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.
677	FAIL-004	modality	90%	MUST send these messages in alignment with a record boundary.	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with
678	FAIL-004	modality	90%	Implementations MUST NOT send zero-length fragments of Handshake	multiple alert messages MUST NOT be coalesced into a single
679	FAIL-004	modality	90%	Implementations MUST NOT send zero-length fragments of Handshake	legacy_record_version: MUST be set to 0x0303 for all records
680	FAIL-004	modality	90%	Implementations MUST NOT send zero-length fragments of Handshake	field is deprecated and MUST be ignored for all purposes.
681	FAIL-004	modality	90%	Implementations MUST NOT send zero-length fragments of Handshake	TLSPplaintext.fragment. The length MUST NOT exceed 2^{14} bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.
682	FAIL-004	modality	90%	Implementations MUST NOT send zero-length fragments of Handshake	ClientHello or a ServerHello MUST have version 0x0303 (reflecting)
683	FAIL-004	modality	90%	Implementations MUST NOT send zero-length fragments of Handshake	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with
684	FAIL-004	modality	90%	Implementations MUST NOT send zero-length fragments of Handshake	If the decryption fails, the receiver MUST terminate the connection
685	FAIL-004	modality	90%	Implementations MUST NOT send zero-length fragments of Handshake	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion
686	FAIL-004	modality	90%	multiple alert messages MUST NOT be coalesced into a single	legacy_record_version: MUST be set to 0x0303 for all records
687	FAIL-004	modality	90%	multiple alert messages MUST NOT be coalesced into a single	field is deprecated and MUST be ignored for all purposes.
688	FAIL-004	modality	90%	multiple alert messages MUST NOT be coalesced into a single	TLSPplaintext.fragment. The length MUST NOT exceed 2^{14} bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.
689	FAIL-004	modality	90%	multiple alert messages MUST NOT be coalesced into a single	ClientHello or a ServerHello MUST have version 0x0303 (reflecting)

690	FAIL-004	modality	90%	multiple alert messages MUST NOT be coalesced into a single	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with
691	FAIL-004	modality	90%	multiple alert messages MUST NOT be coalesced into a single	If the decryption fails, the receiver MUST terminate the connection
692	FAIL-004	modality	90%	multiple alert messages MUST NOT be coalesced into a single	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion
693	FAIL-004	modality	90%	multiple alert messages MUST NOT be coalesced into a single	peer with <code>TLSCiphertext.length</code> larger than $2^{14} + 256$ octets MUST terminate the connection with a "record_overflow" alert. This limit
694	FAIL-004	modality	90%	<code>legacy_record_version</code> : MUST be set to 0x0303 for all records	<code>TLSPiphertext.fragment</code> . The length MUST NOT exceed 2^{14} bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.
695	FAIL-004	modality	90%	<code>legacy_record_version</code> : MUST be set to 0x0303 for all records	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with
696	FAIL-004	modality	90%	<code>legacy_record_version</code> : MUST be set to 0x0303 for all records	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion
697	FAIL-004	modality	90%	field is deprecated and MUST be ignored for all purposes.	<code>TLSPiphertext.fragment</code> . The length MUST NOT exceed 2^{14} bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.
698	FAIL-004	modality	90%	field is deprecated and MUST be ignored for all purposes.	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with
699	FAIL-004	modality	90%	field is deprecated and MUST be ignored for all purposes.	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion
700	FAIL-004	modality	90%	<code>TLSPiphertext.fragment</code> . The length MUST NOT exceed 2^{14} bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.	ClientHello or a ServerHello MUST have version 0x0303 (reflecting)
701	FAIL-004	modality	90%	<code>TLSPiphertext.fragment</code> . The length MUST NOT exceed 2^{14} bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with
702	FAIL-004	modality	90%	<code>TLSPiphertext.fragment</code> . The length MUST NOT exceed 2^{14} bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.	If the decryption fails, the receiver MUST terminate the connection
703	FAIL-004	modality	90%	<code>TLSPiphertext.fragment</code> . The length MUST NOT exceed 2^{14} bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion
704	FAIL-004	modality	90%	<code>TLSPiphertext.fragment</code> . The length MUST NOT exceed 2^{14} bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.	peer with <code>TLSCiphertext.length</code> larger than $2^{14} + 256$ octets MUST terminate the connection with a "record_overflow" alert. This limit
705	FAIL-004	modality	90%	<code>TLSPiphertext.fragment</code> . The length MUST NOT exceed 2^{14} bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.	If a TLS implementation would need to wrap a sequence number, it MUST
706	FAIL-004	modality	90%	<code>TLSPiphertext.fragment</code> . The length MUST NOT exceed 2^{14} bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with a "record_overflow" alert.	MUST NOT be used with TLS. The per-record nonce for the AEAD
707	FAIL-004	modality	90%	ClientHello or a ServerHello MUST have version 0x0303 (reflecting)	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with
708	FAIL-004	modality	90%	ClientHello or a ServerHello MUST have version 0x0303 (reflecting)	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion

709	FAIL-004	modality	90%	ClientHello or a ServerHello MUST have version 0x0303 (reflecting)	MUST NOT be used with TLS. The per-record nonce for the AEAD
710	FAIL-004	modality	90%	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with	If the decryption fails, the receiver MUST terminate the connection
711	FAIL-004	modality	90%	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion
712	FAIL-004	modality	90%	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with	peer with TLSCiphertext.length larger than $2^{14} + 256$ octets MUST terminate the connection with a "record_overflow" alert. This limit
713	FAIL-004	modality	90%	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with	If a TLS implementation would need to wrap a sequence number, it MUST
714	FAIL-004	modality	90%	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with	MUST NOT be used with TLS. The per-record nonce for the AEAD
715	FAIL-004	modality	90%	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with	ContentType field before encryption. Implementations MUST set the
716	FAIL-004	modality	90%	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with	MUST NOT send Handshake and Alert records that have a zero-length
717	FAIL-004	modality	90%	MUST NOT exceed $2^{14} + 256$ bytes. An endpoint that receives a record that exceeds this length MUST terminate the connection with	receiving implementation MUST terminate the connection with an
718	FAIL-004	modality	90%	If the decryption fails, the receiver MUST terminate the connection	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion
719	FAIL-004	modality	90%	If the decryption fails, the receiver MUST terminate the connection	MUST NOT be used with TLS. The per-record nonce for the AEAD
720	FAIL-004	modality	90%	If the decryption fails, the receiver MUST terminate the connection	MUST NOT send Handshake and Alert records that have a zero-length
721	FAIL-004	modality	90%	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion	peer with TLSCiphertext.length larger than $2^{14} + 256$ octets MUST terminate the connection with a "record_overflow" alert. This limit
722	FAIL-004	modality	90%	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion	If a TLS implementation would need to wrap a sequence number, it MUST
723	FAIL-004	modality	90%	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion	MUST NOT be used with TLS. The per-record nonce for the AEAD
724	FAIL-004	modality	90%	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion	ContentType field before encryption. Implementations MUST set the
725	FAIL-004	modality	90%	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion	MUST NOT send Handshake and Alert records that have a zero-length
726	FAIL-004	modality	90%	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion	receiving implementation MUST terminate the connection with an
727	FAIL-004	modality	90%	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion	Implementations MUST limit their scanning to the cleartext returned
728	FAIL-004	modality	90%	An AEAD algorithm used in TLS 1.3 MUST NOT produce an expansion	find a non-zero octet in the cleartext, it MUST terminate the
729	FAIL-004	modality	90%	peer with TLSCiphertext.length larger than $2^{14} + 256$ octets MUST terminate the connection with a "record_overflow" alert. This limit	MUST NOT be used with TLS. The per-record nonce for the AEAD
730	FAIL-004	modality	90%	peer with TLSCiphertext.length larger than $2^{14} + 256$ octets MUST terminate the connection with a "record_overflow" alert. This limit	MUST NOT send Handshake and Alert records that have a zero-length
731	FAIL-004	modality	90%	peer with TLSCiphertext.length larger than $2^{14} + 256$ octets MUST terminate the connection with a "record_overflow" alert. This limit	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^{14}

732	FAIL-004	modality	90%	If a TLS implementation would need to wrap a sequence number, it MUST	MUST NOT be used with TLS. The per-record nonce for the AEAD
733	FAIL-004	modality	90%	If a TLS implementation would need to wrap a sequence number, it MUST	MUST NOT send Handshake and Alert records that have a zero-length
734	FAIL-004	modality	90%	If a TLS implementation would need to wrap a sequence number, it MUST	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14
735	FAIL-004	modality	90%	MUST NOT be used with TLS. The per-record nonce for the AEAD	ContentType field before encryption. Implementations MUST set the
736	FAIL-004	modality	90%	MUST NOT be used with TLS. The per-record nonce for the AEAD	MUST NOT send Handshake and Alert records that have a zero-length
737	FAIL-004	modality	90%	MUST NOT be used with TLS. The per-record nonce for the AEAD	receiving implementation MUST terminate the connection with an
738	FAIL-004	modality	90%	MUST NOT be used with TLS. The per-record nonce for the AEAD	Implementations MUST limit their scanning to the cleartext returned
739	FAIL-004	modality	90%	MUST NOT be used with TLS. The per-record nonce for the AEAD	find a non-zero octet in the cleartext, it MUST terminate the
740	FAIL-004	modality	90%	MUST NOT be used with TLS. The per-record nonce for the AEAD	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14
741	FAIL-004	modality	90%	MUST NOT be used with TLS. The per-record nonce for the AEAD	must still be handled at the TLS layer, though. Later documents may
742	FAIL-004	modality	90%	MUST NOT be used with TLS. The per-record nonce for the AEAD	clients MUST forget the secret values and keys established in failed
743	FAIL-004	modality	90%	MUST NOT be used with TLS. The per-record nonce for the AEAD	AlertLevel=fatal and MUST be treated as error alerts when received
744	FAIL-004	modality	90%	ContentType field before encryption. Implementations MUST set the	MUST NOT send Handshake and Alert records that have a zero-length
745	FAIL-004	modality	90%	ContentType field before encryption. Implementations MUST set the	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14
746	FAIL-004	modality	90%	MUST NOT send Handshake and Alert records that have a zero-length	receiving implementation MUST terminate the connection with an
747	FAIL-004	modality	90%	MUST NOT send Handshake and Alert records that have a zero-length	Implementations MUST limit their scanning to the cleartext returned
748	FAIL-004	modality	90%	MUST NOT send Handshake and Alert records that have a zero-length	find a non-zero octet in the cleartext, it MUST terminate the
749	FAIL-004	modality	90%	MUST NOT send Handshake and Alert records that have a zero-length	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14
750	FAIL-004	modality	90%	MUST NOT send Handshake and Alert records that have a zero-length	must still be handled at the TLS layer, though. Later documents may
751	FAIL-004	modality	90%	MUST NOT send Handshake and Alert records that have a zero-length	clients MUST forget the secret values and keys established in failed
752	FAIL-004	modality	90%	MUST NOT send Handshake and Alert records that have a zero-length	AlertLevel=fatal and MUST be treated as error alerts when received
753	FAIL-004	modality	90%	MUST NOT send Handshake and Alert records that have a zero-length	length) MUST terminate the connection with a "decode_error" alert.
754	FAIL-004	modality	90%	MUST NOT send Handshake and Alert records that have a zero-length	MUST terminate the connection with an "illegal_parameter" alert.
755	FAIL-004	modality	90%	receiving implementation MUST terminate the connection with an	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14
756	FAIL-004	modality	90%	Implementations MUST limit their scanning to the cleartext returned	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14
757	FAIL-004	modality	90%	find a non-zero octet in the cleartext, it MUST terminate the	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14
758	FAIL-004	modality	90%	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14	must still be handled at the TLS layer, though. Later documents may
759	FAIL-004	modality	90%	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14	clients MUST forget the secret values and keys established in failed

760	FAIL-004	modality	90%	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14	AlertLevel=fatal and MUST be treated as error alerts when received
761	FAIL-004	modality	90%	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14	length) MUST terminate the connection with a "decode_error" alert.
762	FAIL-004	modality	90%	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14	MUST terminate the connection with an "illegal_parameter" alert.
763	FAIL-004	modality	90%	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14	The client and the server must share knowledge that the connection is
764	FAIL-004	modality	90%	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14	close_notify: This alert notifies the recipient that the sender will not send any more messages on this connection. Any data received after a closure alert has been received MUST be ignored.
765	FAIL-004	modality	90%	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14	alert has been received MUST be ignored. If a transport-level close
766	FAIL-004	modality	90%	limitations: the full encoded TLSInnerPlaintext MUST NOT exceed 2^14	Each party MUST send a "close_notify" alert before closing its write
767	FAIL-004	modality	90%	deployments will maintain state at that level. Therefore, in normal operation, clients will not know which, if any, of these mechanisms servers actually implement and hence MUST only send early data which	also implement data stores with false positives, such as Bloom filters, in which case they MUST respond to apparent replay by rejecting 0-RTT but MUST NOT abort the handshake.
768	FAIL-004	modality	90%	only a limited number of times. The server MUST ensure that any	also implement data stores with false positives, such as Bloom filters, in which case they MUST respond to apparent replay by rejecting 0-RTT but MUST NOT abort the handshake.
769	FAIL-004	modality	90%	also implement data stores with false positives, such as Bloom filters, in which case they MUST respond to apparent replay by rejecting 0-RTT but MUST NOT abort the handshake.	A TLS-compliant application MUST support digital signatures with
770	FAIL-004	modality	90%	also implement data stores with false positives, such as Bloom filters, in which case they MUST respond to apparent replay by rejecting 0-RTT but MUST NOT abort the handshake.	TLS-compliant application MUST support key exchange with secp256r1
771	FAIL-004	modality	90%	also implement data stores with false positives, such as Bloom filters, in which case they MUST respond to apparent replay by rejecting 0-RTT but MUST NOT abort the handshake.	In the absence of an application profile standard specifying otherwise, a TLS-compliant application MUST implement the following
772	FAIL-004	modality	90%	also implement data stores with false positives, such as Bloom filters, in which case they MUST respond to apparent replay by rejecting 0-RTT but MUST NOT abort the handshake.	All implementations MUST send and use these extensions when offering
773	FAIL-004	modality	90%	- If containing a "supported_groups" extension, it MUST also contain	understand MUST NOT process any messages beyond that ClientHello.
774	FAIL-004	modality	90%	requirements MUST abort the handshake with a "missing_extension"	understand MUST NOT process any messages beyond that ClientHello.
775	FAIL-004	modality	90%	Additionally, all implementations MUST support the use of the	understand MUST NOT process any messages beyond that ClientHello.
776	FAIL-004	modality	90%	MUST follow. It also applies to earlier versions of TLS.	understand MUST NOT process any messages beyond that ClientHello.
777	FAIL-004	modality	90%	For this to work, implementations MUST correctly handle extensible	understand MUST NOT process any messages beyond that ClientHello.
778	FAIL-004	modality	90%	- A client sending a ClientHello MUST support all parameters	understand MUST NOT process any messages beyond that ClientHello.
779	FAIL-004	modality	90%	- A server receiving a ClientHello MUST correctly ignore all	understand MUST NOT process any messages beyond that ClientHello.
780	FAIL-004	modality	90%	- A server receiving a ClientHello MUST correctly ignore all	TLS 1.3 implementations MUST NOT send them but might receive them
781	FAIL-004	modality	90%	- A middlebox which terminates a TLS connection MUST behave as a compliant TLS server (to the original client), including having a	understand MUST NOT process any messages beyond that ClientHello.

782	FAIL-004	modality	90%	- A middlebox which terminates a TLS connection MUST behave as a compliant TLS server (to the original client), including having a	TLS 1.3 implementations MUST NOT send them but might receive them
783	FAIL-004	modality	90%	- A middlebox which terminates a TLS connection MUST behave as a compliant TLS server (to the original client), including having a	versions of TLS and MUST NOT be offered or negotiated by TLS 1.3
784	FAIL-004	modality	90%	compliant TLS client (to the original server), including verifying the original server's certificate. In particular, it MUST generate its own ClientHello containing only parameters it understands, and it MUST generate a fresh ServerHello random	understand MUST NOT process any messages beyond that ClientHello.
785	FAIL-004	modality	90%	compliant TLS client (to the original server), including verifying the original server's certificate. In particular, it MUST generate its own ClientHello containing only parameters it understands, and it MUST generate a fresh ServerHello random	TLS 1.3 implementations MUST NOT send them but might receive them
786	FAIL-004	modality	90%	compliant TLS client (to the original server), including verifying the original server's certificate. In particular, it MUST generate its own ClientHello containing only parameters it understands, and it MUST generate a fresh ServerHello random	versions of TLS and MUST NOT be offered or negotiated by TLS 1.3
787	FAIL-004	modality	90%	understand MUST NOT process any messages beyond that ClientHello.	It MUST forward all subsequent traffic unmodified. Otherwise, it
788	FAIL-004	modality	90%	understand MUST NOT process any messages beyond that ClientHello.	TLS 1.3 implementations MUST NOT send them but might receive them
789	FAIL-004	modality	90%	understand MUST NOT process any messages beyond that ClientHello.	versions of TLS and MUST NOT be offered or negotiated by TLS 1.3
790	FAIL-004	modality	90%	It MUST forward all subsequent traffic unmodified. Otherwise, it	TLS 1.3 implementations MUST NOT send them but might receive them
791	FAIL-004	modality	90%	It MUST forward all subsequent traffic unmodified. Otherwise, it	versions of TLS and MUST NOT be offered or negotiated by TLS 1.3
792	FAIL-004	modality	90%	TLS 1.3 implementations MUST NOT send them but might receive them	versions of TLS and MUST NOT be offered or negotiated by TLS 1.3
793	FAIL-004	modality	90%	TLS 1.3 implementations MUST NOT send them but might receive them	MUST NOT use TLS in such a way absent explicit configuration or a
794	FAIL-004	modality	90%	versions of TLS and MUST NOT be offered or negotiated by TLS 1.3	MUST NOT use TLS in such a way absent explicit configuration or a
795	FAIL-004	modality	90%	MUST NOT use TLS in such a way absent explicit configuration or a	(or is not acceptable), the client MUST abort the handshake with a
796	FAIL-004	modality	90%	MUST NOT use TLS in such a way absent explicit configuration or a	"supported_versions" extension is present, the server MUST negotiate
797	FAIL-004	modality	90%	MUST NOT use TLS in such a way absent explicit configuration or a	"supported_versions" extension is not present, the server MUST negotiate the minimum of ClientHello.legacy_version and TLS 1.2. For
798	FAIL-004	modality	90%	MUST NOT use TLS in such a way absent explicit configuration or a	ClientHello.legacy_version, the server MUST abort the handshake with
799	FAIL-004	modality	90%	MUST NOT use TLS in such a way absent explicit configuration or a	TLS 1.x versions in this field, but its value MUST always be ignored.
800	FAIL-004	modality	90%	(or is not acceptable), the client MUST abort the handshake with a	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.
801	FAIL-004	modality	90%	(or is not acceptable), the client MUST abort the handshake with a	Ciphers with a strength less than 112 bits MUST NOT be offered or
802	FAIL-004	modality	90%	"supported_versions" extension is present, the server MUST negotiate	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.
803	FAIL-004	modality	90%	"supported_versions" extension is present, the server MUST negotiate	Ciphers with a strength less than 112 bits MUST NOT be offered or

804	FAIL-004	modality	90%	"supported_versions" extension is present, the server MUST negotiate	reasons enumerated in [RFC7568], and it MUST NOT be negotiated for
805	FAIL-004	modality	90%	"supported_versions" extension is not present, the server MUST negotiate the minimum of ClientHello.legacy_version and TLS 1.2. For	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.
806	FAIL-004	modality	90%	"supported_versions" extension is not present, the server MUST negotiate the minimum of ClientHello.legacy_version and TLS 1.2. For	Ciphers with a strength less than 112 bits MUST NOT be offered or
807	FAIL-004	modality	90%	"supported_versions" extension is not present, the server MUST negotiate the minimum of ClientHello.legacy_version and TLS 1.2. For	reasons enumerated in [RFC7568], and it MUST NOT be negotiated for
808	FAIL-004	modality	90%	"supported_versions" extension is not present, the server MUST negotiate the minimum of ClientHello.legacy_version and TLS 1.2. For	reasons enumerated in [RFC6176], and it MUST NOT be negotiated for
809	FAIL-004	modality	90%	legacy_version is TLS 1.0, the server will proceed with a TLS 1.0	Ciphers with a strength less than 112 bits MUST NOT be offered or
810	FAIL-004	modality	90%	ClientHello.legacy_version, the server MUST abort the handshake with	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.
811	FAIL-004	modality	90%	ClientHello.legacy_version, the server MUST abort the handshake with	Ciphers with a strength less than 112 bits MUST NOT be offered or
812	FAIL-004	modality	90%	ClientHello.legacy_version, the server MUST abort the handshake with	reasons enumerated in [RFC7568], and it MUST NOT be negotiated for
813	FAIL-004	modality	90%	ClientHello.legacy_version, the server MUST abort the handshake with	reasons enumerated in [RFC6176], and it MUST NOT be negotiated for
814	FAIL-004	modality	90%	ClientHello.legacy_version, the server MUST abort the handshake with	Implementations MUST NOT send an SSL version 2.0 compatible
815	FAIL-004	modality	90%	ClientHello.legacy_version, the server MUST abort the handshake with	CLIENT-HELLO. Implementations MUST NOT negotiate TLS 1.3 or later
816	FAIL-004	modality	90%	TLS 1.x versions in this field, but its value MUST always be ignored.	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.
817	FAIL-004	modality	90%	TLS 1.x versions in this field, but its value MUST always be ignored.	Ciphers with a strength less than 112 bits MUST NOT be offered or
818	FAIL-004	modality	90%	TLS 1.x versions in this field, but its value MUST always be ignored.	reasons enumerated in [RFC7568], and it MUST NOT be negotiated for
819	FAIL-004	modality	90%	TLS 1.x versions in this field, but its value MUST always be ignored.	reasons enumerated in [RFC6176], and it MUST NOT be negotiated for
820	FAIL-004	modality	90%	TLS 1.x versions in this field, but its value MUST always be ignored.	Implementations MUST NOT send an SSL version 2.0 compatible
821	FAIL-004	modality	90%	TLS 1.x versions in this field, but its value MUST always be ignored.	CLIENT-HELLO. Implementations MUST NOT negotiate TLS 1.3 or later
822	FAIL-004	modality	90%	will respond to the ClientHello with an older ServerHello, but it will not correctly skip the 0-RTT data and will fail to complete the	Ciphers with a strength less than 112 bits MUST NOT be offered or
823	FAIL-004	modality	90%	A client that attempts to send 0-RTT data MUST fail a connection if	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.
824	FAIL-004	modality	90%	A client that attempts to send 0-RTT data MUST fail a connection if	Ciphers with a strength less than 112 bits MUST NOT be offered or
825	FAIL-004	modality	90%	A client that attempts to send 0-RTT data MUST fail a connection if	reasons enumerated in [RFC7568], and it MUST NOT be negotiated for
826	FAIL-004	modality	90%	A client that attempts to send 0-RTT data MUST fail a connection if	reasons enumerated in [RFC6176], and it MUST NOT be negotiated for
827	FAIL-004	modality	90%	A client that attempts to send 0-RTT data MUST fail a connection if	Implementations MUST NOT send an SSL version 2.0 compatible
828	FAIL-004	modality	90%	A client that attempts to send 0-RTT data MUST fail a connection if	CLIENT-HELLO. Implementations MUST NOT negotiate TLS 1.3 or later

829	FAIL-004	modality	90%	A client that attempts to send 0-RTT data MUST fail a connection if	Implementations MUST NOT send any records with a version less than
830	FAIL-004	modality	90%	change_cipher_spec at any time during the handshake, as they must be	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.
831	FAIL-004	modality	90%	change_cipher_spec at any time during the handshake, as they must be	Ciphers with a strength less than 112 bits MUST NOT be offered or
832	FAIL-004	modality	90%	change_cipher_spec at any time during the handshake, as they must be	reasons enumerated in [RFC7568], and it MUST NOT be negotiated for
833	FAIL-004	modality	90%	change_cipher_spec at any time during the handshake, as they must be	reasons enumerated in [RFC6176], and it MUST NOT be negotiated for
834	FAIL-004	modality	90%	change_cipher_spec at any time during the handshake, as they must be	Implementations MUST NOT send an SSL version 2.0 compatible
835	FAIL-004	modality	90%	change_cipher_spec at any time during the handshake, as they must be	CLIENT-HELLO. Implementations MUST NOT negotiate TLS 1.3 or later
836	FAIL-004	modality	90%	change_cipher_spec at any time during the handshake, as they must be	Implementations MUST NOT send any records with a version less than
837	FAIL-004	modality	90%	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.	Ciphers with a strength less than 112 bits MUST NOT be offered or
838	FAIL-004	modality	90%	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.	reasons enumerated in [RFC7568], and it MUST NOT be negotiated for
839	FAIL-004	modality	90%	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.	reasons enumerated in [RFC6176], and it MUST NOT be negotiated for
840	FAIL-004	modality	90%	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.	Implementations MUST NOT send an SSL version 2.0 compatible
841	FAIL-004	modality	90%	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.	CLIENT-HELLO. Implementations MUST NOT negotiate TLS 1.3 or later
842	FAIL-004	modality	90%	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.	ServerHello.legacy_version set to 0x0300 MUST abort the handshake
843	FAIL-004	modality	90%	reasons cited in [RFC7465]. Implementations MUST NOT offer or negotiate RC4 cipher suites for any version of TLS for any reason.	Implementations MUST NOT send any records with a version less than
844	FAIL-004	modality	90%	Ciphers with a strength less than 112 bits MUST NOT be offered or	reasons enumerated in [RFC7568], and it MUST NOT be negotiated for
845	FAIL-004	modality	90%	Ciphers with a strength less than 112 bits MUST NOT be offered or	reasons enumerated in [RFC6176], and it MUST NOT be negotiated for
846	FAIL-004	modality	90%	Ciphers with a strength less than 112 bits MUST NOT be offered or	Implementations MUST NOT send an SSL version 2.0 compatible
847	FAIL-004	modality	90%	Ciphers with a strength less than 112 bits MUST NOT be offered or	CLIENT-HELLO. Implementations MUST NOT negotiate TLS 1.3 or later
848	FAIL-004	modality	90%	Ciphers with a strength less than 112 bits MUST NOT be offered or	ServerHello.legacy_version set to 0x0300 MUST abort the handshake
849	FAIL-004	modality	90%	Ciphers with a strength less than 112 bits MUST NOT be offered or	Implementations MUST NOT send any records with a version less than
850	FAIL-004	modality	90%	reasons enumerated in [RFC7568], and it MUST NOT be negotiated for	reasons enumerated in [RFC6176], and it MUST NOT be negotiated for
851	FAIL-004	modality	90%	reasons enumerated in [RFC7568], and it MUST NOT be negotiated for	Implementations MUST NOT send an SSL version 2.0 compatible
852	FAIL-004	modality	90%	reasons enumerated in [RFC7568], and it MUST NOT be negotiated for	CLIENT-HELLO. Implementations MUST NOT negotiate TLS 1.3 or later
853	FAIL-004	modality	90%	reasons enumerated in [RFC7568], and it MUST NOT be negotiated for	ServerHello.legacy_version set to 0x0300 MUST abort the handshake

854	FAIL-004	modality	90%	reasons enumerated in [RFC7568], and it MUST NOT be negotiated for	Implementations MUST NOT send any records with a version less than
855	FAIL-004	modality	90%	reasons enumerated in [RFC6176], and it MUST NOT be negotiated for	Implementations MUST NOT send an SSL version 2.0 compatible
856	FAIL-004	modality	90%	reasons enumerated in [RFC6176], and it MUST NOT be negotiated for	CLIENT-HELLO. Implementations MUST NOT negotiate TLS 1.3 or later
857	FAIL-004	modality	90%	reasons enumerated in [RFC6176], and it MUST NOT be negotiated for	ServerHello.legacy_version set to 0x0300 MUST abort the handshake
858	FAIL-004	modality	90%	reasons enumerated in [RFC6176], and it MUST NOT be negotiated for	Implementations MUST NOT send any records with a version less than
859	FAIL-004	modality	90%	reasons enumerated in [RFC6176], and it MUST NOT be negotiated for	MUST NOT combine external PSKs with certificate-based authentication
860	FAIL-004	modality	90%	Implementations MUST NOT send an SSL version 2.0 compatible	CLIENT-HELLO. Implementations MUST NOT negotiate TLS 1.3 or later
861	FAIL-004	modality	90%	Implementations MUST NOT send an SSL version 2.0 compatible	ServerHello.legacy_version set to 0x0300 MUST abort the handshake
862	FAIL-004	modality	90%	Implementations MUST NOT send an SSL version 2.0 compatible	Implementations MUST NOT send any records with a version less than
863	FAIL-004	modality	90%	Implementations MUST NOT send an SSL version 2.0 compatible	MUST NOT combine external PSKs with certificate-based authentication
864	FAIL-004	modality	90%	Implementations MUST NOT send an SSL version 2.0 compatible	to be used as channel bindings, the exported value MUST be large
865	FAIL-004	modality	90%	CLIENT-HELLO. Implementations MUST NOT negotiate TLS 1.3 or later	ServerHello.legacy_version set to 0x0300 MUST abort the handshake
866	FAIL-004	modality	90%	CLIENT-HELLO. Implementations MUST NOT negotiate TLS 1.3 or later	Implementations MUST NOT send any records with a version less than
867	FAIL-004	modality	90%	CLIENT-HELLO. Implementations MUST NOT negotiate TLS 1.3 or later	MUST NOT combine external PSKs with certificate-based authentication
868	FAIL-004	modality	90%	CLIENT-HELLO. Implementations MUST NOT negotiate TLS 1.3 or later	to be used as channel bindings, the exported value MUST be large
869	FAIL-004	modality	90%	ServerHello.legacy_version set to 0x0300 MUST abort the handshake	Implementations MUST NOT send any records with a version less than
870	FAIL-004	modality	90%	ServerHello.legacy_version set to 0x0300 MUST abort the handshake	MUST NOT combine external PSKs with certificate-based authentication
871	FAIL-004	modality	90%	Implementations MUST NOT send any records with a version less than	MUST NOT combine external PSKs with certificate-based authentication
872	FAIL-004	modality	90%	Implementations MUST NOT send any records with a version less than	to be used as channel bindings, the exported value MUST be large
873	FAIL-004	modality	90%	MUST NOT combine external PSKs with certificate-based authentication	to be used as channel bindings, the exported value MUST be large
874	FAIL-004	modality	90%	to be used as channel bindings, the exported value MUST be large	Application protocols MUST NOT use 0-RTT data without a profile that
875	FAIL-004	modality	90%	Application protocols MUST NOT use 0-RTT data without a profile that	In addition, to avoid accidental misuse, TLS implementations MUST NOT enable 0-RTT (either sending or accepting) unless specifically requested by the application and MUST NOT automatically resend 0-RTT
876	FAIL-004	modality	90%	Application protocols MUST NOT use 0-RTT data without a profile that	processing, TLS implementations MUST provide a way for the
877	FAIL-004	modality	90%	Application protocols MUST NOT use 0-RTT data without a profile that	both TLS 1.2 and TLS 1.3 must be used with only one hash in TLS 1.3,
878	FAIL-004	modality	90%	In addition, to avoid accidental misuse, TLS implementations MUST NOT enable 0-RTT (either sending or accepting) unless specifically requested by the application and MUST NOT automatically resend 0-RTT	processing, TLS implementations MUST provide a way for the

879	FAIL-004	modality	90%	In addition, to avoid accidental misuse, TLS implementations MUST NOT enable 0-RTT (either sending or accepting) unless specifically requested by the application and MUST NOT automatically resend 0-RTT	both TLS 1.2 and TLS 1.3 must be used with only one hash in TLS 1.3,
880	FAIL-003	semantic	90%	certificates. The client MUST check EncryptedExtensions for the presence of any forbidden extensions and if any are found MUST abort	The client MUST send a Certificate message if and only if the server
881	FAIL-003	semantic	90%	extension to the client. Clients MUST NOT act upon any information	correspond to a group which was provided in the "key_share" extension
882	FAIL-003	semantic	90%	transmitted by the client. The server MUST generate a fresh value	messages for a given response MUST appear consecutively on the wire
883	FAIL-003	semantic	89%	the client MUST send an EndOfEarlyData message after receiving the	transmitted by the client. The server MUST generate a fresh value
884	FAIL-003	semantic	88%	receives any other value, it MUST terminate the connection with an	ClientHello message or after the peer's Finished message, it MUST be
885	FAIL-003	semantic	88%	0x0303 (TLS 1.2). Clients MUST check for this extension prior to processing the rest of the ServerHello (although they will have to	ClientHello). When sending the new ClientHello, the client MUST copy
886	FAIL-003	semantic	87%	(or is not acceptable), the client MUST abort the handshake with a	"supported_versions" extension is not present, the server MUST negotiate the minimum of ClientHello.legacy_version and TLS 1.2. For
887	FAIL-003	semantic	86%	The client MUST send a Certificate message if and only if the server	well. Extensions in the Certificate message from the server MUST correspond to ones from the ClientHello message. Extensions in the Certificate message from the client MUST correspond to
888	FAIL-003	semantic	86%	validate using any signature algorithm using an MD5 hash MUST abort	The receiver of a CertificateVerify message MUST verify the signature
889	FAIL-003	semantic	86%	For externally established PSKs, the Hash algorithm MUST be set when	value is not present or does not validate, the server MUST abort the
890	FAIL-003	semantic	84%	"supported_versions" extension is not present, the server MUST negotiate the minimum of ClientHello.legacy_version and TLS 1.2. For	A client that attempts to send 0-RTT data MUST fail a connection if
891	FAIL-003	semantic	84%	Clients MUST verify that the server's selected_identity is within the	Servers MUST check that it is the last extension and otherwise fail
892	FAIL-003	semantic	84%	response to a ClientHello MUST set the last 8 bytes of their Random	and which receives a TLS 1.3 ServerHello during renegotiation MUST
893	FAIL-003	semantic	84%	The receiver of a CertificateVerify message MUST verify the signature	Recipients of Finished messages MUST verify that the contents are correct and if incorrect MUST terminate the connection with a
894	FAIL-003	semantic	83%	authenticated, this has to be provisioned by the application layer.	(which processes the data immediately) and another cluster which will fall back to 1-RTT and process the data upon application-layer
895	FAIL-003	semantic	82%	inherent replay protections for 0-RTT data. There are two potential	still require consistent storage for anti-replay of 0-RTT data, as
896	FAIL-003	semantic	81%	negotiation, then this will happen automatically). Upon receiving the ServerHello, clients MUST check that the cipher suite supplied in	"supported_versions" extension MUST be retained in the ServerHello, and a client MUST abort the handshake with an "illegal_parameter"
897	FAIL-003	semantic	81%	early data MUST be the first PSK listed in the client's	In order to accept early data, the server MUST have accepted a PSK
898	FAIL-003	semantic	81%	ClientHello). When sending the new ClientHello, the client MUST copy	extension, then the server MUST abort the handshake with a
899	FAIL-003	semantic	80%	- When (EC)DHE is in use, the server will also provide a "key_share"	ClientHello as its first TLS message. The client will also send a

5. S2 Taxonomy Findings

Total findings: **159**

Code	Description	Severity	Count
Ambiguous Referent		—	56
Hidden Normative		—	61
Missing Temporal Anchor		—	32
Modal Inconsistency		—	4
Vague Qualifier		—	6

All Findings

Code	Sev.	Category	Text Span	Explanation
Modal Inconsistency	Medium	Logical	"MUST" vs "may"	Shared topic (byte, empty) is governed by an obligation modal in one statement and a permission modal in another, potentially creating a compliance loophole.
Modal Inconsistency	Medium	Logical	"MUST" vs "may"	Shared topic (accepted, data) is governed by an obligation modal in one statement and a permission modal in another, potentially creating a compliance loophole.
Modal Inconsistency	Medium	Logical	"MUST" vs "may"	Shared topic (data, which) is governed by an obligation modal in one statement and a permission modal in another, potentially creating a compliance loophole.
Modal Inconsistency	Medium	Logical	"require" vs "may"	Shared topic (data, replay, storage) is governed by an obligation modal in one statement and a permission modal in another, potentially creating a compliance loophole.
Hidden Normative	Medium	Context	should be	Normative-strength language ("should be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should not	Normative-strength language ("should not") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD NOT	Normative-strength language ("SHOULD NOT") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should not	Normative-strength language ("should not") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD be	Normative-strength language ("SHOULD be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD NOT	Normative-strength language ("SHOULD NOT") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should be	Normative-strength language ("should be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.

Hidden Normative	Medium	Context	it is RECOMMENDED that	Normative-strength language ("it is RECOMMENDED that") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD only	Normative-strength language ("SHOULD only") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	is expected to	Normative-strength language ("is expected to") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD NOT	Normative-strength language ("SHOULD NOT") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	It is RECOMMENDED that	Normative-strength language ("It is RECOMMENDED that") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should not	Normative-strength language ("should not") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD be	Normative-strength language ("SHOULD be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD be	Normative-strength language ("SHOULD be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should be	Normative-strength language ("should be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should never	Normative-strength language ("should never") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should never	Normative-strength language ("should never") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should never	Normative-strength language ("should never") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should never	Normative-strength language ("should never") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD be	Normative-strength language ("SHOULD be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should be	Normative-strength language ("should be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.

Hidden Normative	Medium	Context	should be	Normative-strength language ("should be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should be	Normative-strength language ("should be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should always	Normative-strength language ("should always") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	It is RECOMMENDED that	Normative-strength language ("It is RECOMMENDED that") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD NOT	Normative-strength language ("SHOULD NOT") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	all implementations SHOULD	Normative-strength language ("all implementations SHOULD") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD NOT	Normative-strength language ("SHOULD NOT") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD NOT	Normative-strength language ("SHOULD NOT") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD NOT	Normative-strength language ("SHOULD NOT") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should be	Normative-strength language ("should be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should be	Normative-strength language ("should be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should not	Normative-strength language ("should not") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should be	Normative-strength language ("should be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should always	Normative-strength language ("should always") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD NOT	Normative-strength language ("SHOULD NOT") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.

Hidden Normative	Medium	Context	it is necessary that	Normative-strength language ("it is necessary that") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should be	Normative-strength language ("should be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD be	Normative-strength language ("SHOULD be") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should not	Normative-strength language ("should not") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should not	Normative-strength language ("should not") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should not	Normative-strength language ("should not") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should not	Normative-strength language ("should not") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	should not	Normative-strength language ("should not") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Hidden Normative	Medium	Context	SHOULD NOT	Normative-strength language ("SHOULD NOT") appears in a segment classified as informative. Requirements embedded in informative sections may be missed during compliance review.
Vague Qualifier	Low	Vagueness	appropriate	Vague qualifier "appropriate" in a normative statement creates a non-verifiable compliance threshold.
Vague Qualifier	Low	Vagueness	appropriate	Vague qualifier "appropriate" in a normative statement creates a non-verifiable compliance threshold.
Vague Qualifier	Low	Vagueness	generally	Vague qualifier "generally" in a normative statement creates a non-verifiable compliance threshold.
Vague Qualifier	Low	Vagueness	appropriate	Vague qualifier "appropriate" in a normative statement creates a non-verifiable compliance threshold.
Vague Qualifier	Low	Vagueness	appropriate	Vague qualifier "appropriate" in a normative statement creates a non-verifiable compliance threshold.
Vague Qualifier	Low	Vagueness	appropriate	Vague qualifier "appropriate" in a normative statement creates a non-verifiable compliance threshold.
Ambiguous Referent	Medium	Vagueness	that is	Pronoun "that" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	it may	Pronoun "it" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.

[illegible]

[illegible]

Ambiguous Referent	Medium	Vagueness	it will	Pronoun "it" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	they MUST	Pronoun "they" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	it will	Pronoun "it" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	that might	Pronoun "that" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	it MUST	Pronoun "it" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	it MUST	Pronoun "it" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	it MUST	Pronoun "it" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	It MUST	Pronoun "It" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	it is	Pronoun "it" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	it will	Pronoun "it" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	it will	Pronoun "it" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	they must	Pronoun "they" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	it MUST	Pronoun "it" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Ambiguous Referent	Medium	Vagueness	it MUST	Pronoun "it" in a normative statement has an ambiguous referent — the requirement subject cannot be determined without additional context.
Missing Temporal Anchor	High	Logical	alert	Time-sensitive verb "alert" in a normative statement with no specific deadline or temporal qualifier (e.g., "within N days", "no later than").
Missing Temporal Anchor	High	Logical	alert	Time-sensitive verb "alert" in a normative statement with no specific deadline or temporal qualifier (e.g., "within N days", "no later than").
Missing Temporal Anchor	High	Logical	alert	Time-sensitive verb "alert" in a normative statement with no specific deadline or temporal qualifier (e.g., "within N days", "no later than").
Missing Temporal Anchor	High	Logical	alert	Time-sensitive verb "alert" in a normative statement with no specific deadline or temporal qualifier (e.g., "within N days", "no later than").
Missing Temporal Anchor	High	Logical	respond	Time-sensitive verb "respond" in a normative statement with no specific deadline or temporal qualifier (e.g., "within N days", "no later than").
Missing Temporal Anchor	High	Logical	alert	Time-sensitive verb "alert" in a normative statement with no specific deadline or temporal qualifier (e.g., "within N days", "no later than").

Missing Temporal Anchor	High	Logical	alert	Time-sensitive verb "alert" in a normative statement with no specific deadline or temporal qualifier (e.g., "within N days", "no later than").
Missing Temporal Anchor	High	Logical	alert	Time-sensitive verb "alert" in a normative statement with no specific deadline or temporal qualifier (e.g., "within N days", "no later than").
Missing Temporal Anchor	High	Logical	respond	Time-sensitive verb "respond" in a normative statement with no specific deadline or temporal qualifier (e.g., "within N days", "no later than").
Missing Temporal Anchor	High	Logical	forward	Time-sensitive verb "forward" in a normative statement with no specific deadline or temporal qualifier (e.g., "within N days", "no later than").
Missing Temporal Anchor	High	Logical	produce	Time-sensitive verb "produce" in a normative statement with no specific deadline or temporal qualifier (e.g., "within N days", "no later than").
Missing Temporal Anchor	High	Logical	provide	Time-sensitive verb "provide" in a normative statement with no specific deadline or temporal qualifier (e.g., "within N days", "no later than").